

Rapport projet PAF



David VADIMON

Mai 2026

Table des matières

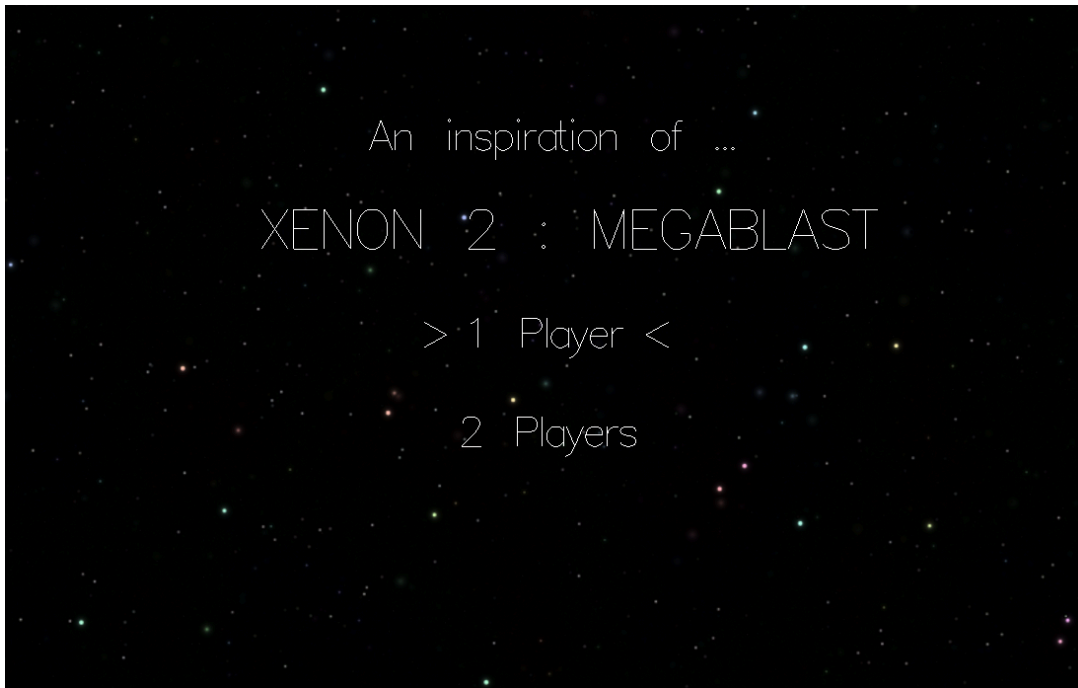
Table des matières	2
Manuel d'utilisation et fonctionnalités	5
Liste exhaustive des propriétés	8
Keyboard.hs	8
Pré-conditions	8
Typeclasses/Movable.hs	8
Lois	8
Typeclasses/Invariant.hs	8
Lois	8
Typeclasses/Destroyable.hs	8
Lois	8
Typeclasses/Damageable.hs	9
Lois	9
Objects/Hitbox.hs	9
Invariants	9
Post-conditions	9
Propriétés algébriques	9
Objects/Objects.hs	10
Lois	10
Invariants	10
Pré-conditions	10
Post-conditions	10
Propriétés algébriques	10
Graphics/Assets.hs	11
Lois	11
Pré-conditions	11
Post-conditions	11
Graphics/Background.hs	11
Invariants	11
Pré-conditions	12
Post-conditions	12
Graphics/Explosion.hs	12
Invariants	12
Pré-conditions	12
Post-conditions	12
GameState/Rock.hs	12
Invariants	12
Pré-conditions	12
Post-conditions	13
GameState/Wall.hs	13
Invariants	13
Pré-conditions	13

Post-conditions	13
Propriétés	14
GameState/Projectile.hs	14
Invariants	14
Pré-conditions	14
Post-conditions	14
GameState/Bonus.hs	14
Invariants	14
Pré-conditions	15
Post-conditions	15
GameState/Player.hs	15
Invariants	15
Pré-conditions	15
Post-conditions	16
GameState/Enemy.hs	17
Invariants	17
Pré-conditions	17
Post-conditions	17
GameState/Game.hs	18
Invariants	18
Pré-conditions	18
Post-conditions	19
Description des tests	21
Méthodologie générale des tests	21
Générateurs et constructeurs	21
Test unitaires et QuickCheck	24
Vérification des lois	26
Point pertinent : génération de Hitboxes	27
Point pertinent : génération des murs finis et infinis	28
Description du projet	33
Schéma des classes de types et types de données (à partir du répertoire src)	33
Description globale de l'architecture	34
Description des classes de types et types de données	35
GameSetup.hs	35
Keyboard.hs	35
Typeclasses/Damageable.hs	36
Typeclasses/Destroyable.hs	37
Typeclasses/Movable.hs	37
Typeclasses/Invariant.hs	38
Objects/Hitbox.hs	39
Objects/Objects.hs	39
Graphics/Assets.hs	41
Graphics/Background.hs	41
Graphics/Explosion.hs	42

GameState/Rock.hs	43
GameState/Wall.hs	43
GameState/Projectile.hs	44
GameState/Bonus.hs	44
GameState/Player.hs	44
GameState/Enemy.hs	45
GameState/Game.hs	46
Boucle principale du jeu	47
Extensions implémentées	49
Extensions basiques	49
Point pertinent : gestion du repoussement par un mur	49
Point pertinent : suivi du comportement d'un ennemi	50
Extensions avancées	51
Génération procédurale du jeu	51
Jeu à 2 joueurs coopératif	52
Invincibilité temporaire sur perte de vie	53
Bonus de tir pour le joueur	55

Manuel d'utilisation et fonctionnalités

Le jeu se lance par le lancement de la commande **stack run** à la racine du projet.
L'écran d'accueil est alors affiché :



L'utilisateur peut selon les touches :

- Appuyer sur **Z / S**, ou sur les flèches **HAUT / BAS**, pour sélectionner l'option "1 Player" ou "2 Players".
- Appuyer sur la touche **ESPACE** ou **ENTRÉE** pour lancer le jeu avec l'option sélectionnée.
- Appuyer sur la touche **ECHAP** pour fermer le jeu.

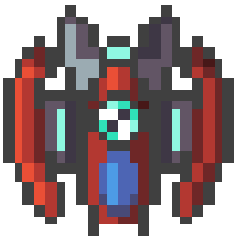
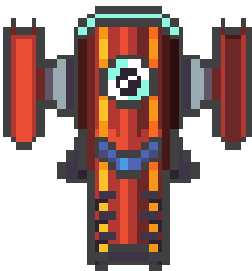
Une fois **en jeu** :



- Appuyer sur les touches **Z / Q / S / D** permet au vaisseau du **joueur 1** de se déplacer dans la direction souhaitée, et **ESPACE** lui permet de tirer devant lui.
- Appuyer sur les flèches **HAUT / GAUCHE / BAS / DROITE** permet au vaisseau du **joueur 2**, de se déplacer dans la direction souhaitée, et **ENTRÉE** lui permet de tirer devant lui.
- Appuyer sur la touche **ECHAP** permet de revenir à l'écran d'accueil.

Chaque joueur possède **3 vies**, chacune ayant **100 points de vie**, et en perd à chaque collision avec un adversaire ou avec un de ses projectiles. Les mouvements du joueur sont restreints à l'écran, et il ne lui est pas possible de rentrer en collision avec un mur, ou avec l'autre joueur, si celui-ci est présent. De même, les murs peuvent repousser le joueur si celui-ci est placé dans une position délicate.

Le déroulement du jeu ne comporte **pas de script global** : de manière illimitée et périodique, des murs flottants finis ainsi que des ennemis par vague vont apparaître aléatoirement face au joueur afin de lui compliquer la tâche, donnant une sorte de **génération procédurale**. Chaque ennemi possède un comportement défini, via son **script interne**, ainsi que des caractéristiques uniques :

	Le NoMoveButBoomEnemy, celui-ci ne fait qu'être défilé vers le bas, en suivant la vitesse de l'écran. Il possède 1 point de vie, mais inflige 20 points de dégâts au joueur en cas de collision. Sa mort fait gagner 10 points de score à celui le tuant.
	Le LeftRightShootEnemy, celui-ci commence à descendre vers une ordonnée dans l'écran choisie aléatoirement, et se déplace ensuite de gauche à droite, le tout en tirant chaque seconde devant lui. Il possède 1 point de vie, inflige 10 points de dégâts au joueur en cas de collision avec lui ou avec un de ses projectiles tirés. Sa mort vaut 25 points à celui le tuant.
	Le LoopEnemy, celui-ci commence à descendre vers une ordonnée dans l'écran choisie aléatoirement, effectue une spirale complète, et continue sa route vers le bas, tout en ayant une vitesse élevée. Il possède 3 points de vie, inflige 10 points de dégâts au joueur en cas de collision avec lui, mais peut superposer ses dégâts selon sa vie restante : si il lui reste 3 points de vie, une collision avec le joueur inflige alors $3 \times 10 = 30$ points de dégâts. Sa mort vaut 50 points à celui le tuant.

Par défaut les collisions ainsi que les projectiles du joueur infligent **1 point de dégât aux ennemis**, néanmoins il y a **10% de chances** à chaque mort d'un ennemi, tué par un projectile, que celui-ci lâche un **bonus** à son endroit de décès. Il existe 4 bonus, ramassables par un joueur qui passe dessus :

	L'agrandissement de la taille du tir du joueur.
	Le triplement des dégâts du tir d'un joueur.
	La division par 3 du délai du tir du joueur.
	Le triplement de la vitesse du tir du joueur.

Ces bonus ne sont pas cumulables, donc en prendre un remplace l'ancien.

Lorsqu'un joueur perd une vie, si ce n'est pas sa dernière alors il passe en **période d'invincibilité de 2 secondes** (durant lesquelles il clignote en blanc).

Au contraire, lorsqu'un joueur meurt en perdant sa dernière vie, celui-ci ne peut alors plus interagir avec le jeu.

Lorsque tous les joueurs sont morts, le jeu continue son déroulement : pour relancer une partie il faut alors comme dit précédemment appuyer sur **ECHAP** pour revenir à l'écran d'accueil, et alors pouvoir en relancer une nouvelle !

Concernant le lancement des tests, il est possible à partir de la racine du projet :

- De tous les lancer avec la commande **stack test**.
- D'en lancer une sous partie, par exemple avec la commande :

stack test --test-arguments="-m [Game]"

où *Game* peut être remplacé par : *Keyboard, Hitbox, Objects, Assets, Background, Explosion, Rock, Wall, Projectile, Player, Enemy, Bonus*

Liste exhaustive des propriétés

Etant donné que j'ai un nombre assez important de propriétés, je vais essayer d'être concis dans les explications. Tous les noms de fichiers sont donnés à partir du répertoire **src** (aucune propriété n'est dans mon *app/Main.hs*). Seules les signatures sont données, se référer au code source pour les implémentations.

Keyboard.hs

Pré-conditions

```
prop_pre_playerNewDirectionSpeed :: Keyboard -> Float -> Bool
```

> vérifie pour l'opération `playerNewDirectionSpeed` que le flottant donné en paramètre est positif ou nul.

Typeclasses/Movable.hs

Lois

```
law_positive_screenSpeed :: a -> ScreenScrollingSpeed -> Bool
```

> doit être vérifié par une instance de `Movable`, spécifie le fait de devoir recevoir une `ScreenScrollingSpeed` positive ou nulle en paramètre de l'opération `move`.

Typeclasses/Invariant.hs

Lois

```
law_invariant_stable :: Invariant a => a -> Property
```

> doit être vérifié par une instance de `Invariant`, assure que lorsque l'invariant est vrai pour une valeur, le vérifier à nouveau donne le même résultat.

```
law_invariant_idempotent :: Invariant a => a -> Bool
```

> doit être vérifié par une instance de `Invariant`, assure que si l'invariant est vrai une fois, sa vérification reste cohérente.

```
law_invariant_preserved :: (Invariant a, Invariant b) => (a -> b) -> a -> Property
```

> doit être vérifié par une instance de `Invariant`, assure plusieurs fois que l'invariant ne modifie pas son résultat.

Typeclasses/Destroyable.hs

Lois

```
law_destroyable_damage_cumulative :: (Eq a, Destroyable a) => Damage -> Damage -> a -> Bool
```

> doit être vérifié par une instance de `Invariant`, assure que les dégâts reçus par une entité encore en vie sont cumulatifs.

```
law_destroyable_zero_damage_identity :: (Eq a, Destroyable a) => a -> Bool
```

> doit être vérifié par une instance de `Destroyable`, assure que recevoir zéro dégât ne modifie pas l'entité.


```
law_destroyable_no_heal_negative_damage :: (Eq a, Destroyable a) => Damage -> a -> Property
```

> doit être vérifié par une instance de `Destroyable`, assure que des dégâts négatifs ne restaurent pas de points de vie.

Typeclasses/Damageable.hs

Lois

```
law_destroyable_damage_cumulative :: (Eq a, Destroyable a) => Damage -> Damage -> a -> Bool
```

> doit être vérifié par une instance de `Destroyable`, assure qu'une entité morte reste morte après avoir reçu des dégâts.

```
law_destroyable_zero_damage_identity :: (Eq a, Destroyable a) => a -> Bool
```

> doit être vérifié par une instance de `Destroyable`, assure que recevoir des dégâts supplémentaires après la mort ne modifie plus l'entité.

```
law_destroyable_no_heal_negative_damage :: (Eq a, Destroyable a) => Damage -> a -> Property
```

> doit être vérifié par une instance de `Destroyable`, assure que recevoir zéro dégât ne modifie pas l'entité.

```
law_destroyable_no_heal_negative_damage :: (Eq a, Destroyable a) => Damage -> a -> Property
```

> doit être vérifié par une instance de `Destroyable`, assure que des dégâts négatifs ne restaurent pas de points de vie.

Objects/Hitbox.hs

Invariants

```
prop_inv_hitbox :: Hitbox -> Bool
```

> assure qu'un `Circle` a un rayon positif, qu'un `Rectangle` a une largeur et une longueur strictement positives, et qu'un `Hitboxes` contient une liste de `Hitbox` non vide, avec son centre présent dans une `Hitbox` de la liste.

Post-conditions

```
prop_post_centerHitbox :: Hitbox -> Bool
```

> vérifie pour l'opération `centerHitbox` que le couple (`XCenter`, `YCenter`) de coordonnées résultantes font partie de la `Hitbox` donnée en paramètre.

```
prop_post_moveHitbox :: Hitbox -> (XCoord, YCoord) -> Bool
```

> vérifie à l'issue d'un mouvement de `Hitbox`, pour l'opération `moveHitbox`, que les nouvelles coordonnées de la `Hitbox` ont été correctement déplacées.

```
prop_post_clamp :: Ord a => a -> a -> a -> Bool
```

> vérifie que la valeur retournée par l'opération `clamp` est dans l'intervalle voulu.

Propriétés algébriques

```
prop_commutativity_collisionHitbox :: Hitbox -> Hitbox -> Bool
```

> vérifie la commutativité de l'opération collisionHitbox.

Objects/Objects.hs

Lois

law_collidable_reflexive :: Collidable a => a -> Bool

> doit être vérifié par une instance de Collidable, assure qu'un objet doit toujours entrer en collision avec lui-même.

law_collidable_symmetric :: (Collidable a, Collidable b) => a -> b -> Bool

> doit être vérifié par une instance de Collidable, assure que L'ordre des arguments ne change pas la détection de collision.

law_collidable_will_collide :: Collidable a => Collidable b => a -> b -> ScreenScrollingSpeed -> Bool

> doit être vérifié par une instance de Collidable, assure que la prédiction de collision correspond à la collision après déplacement.

Invariants

prop_inv_direction :: Direction -> Bool

> spécifie qu'une direction n'a que des composantes X et Y ayant une valeur dans {-1, 0, 1}.

prop_inv_objectSpeed :: ObjectSpeed -> Bool

> spécifie que la vitesse d'un objet ne peut qu'être positive.

prop_inv_object :: Object -> Bool

> spécifie qu'un objet statique possède une Hitbox, une Direction, et une ObjectSpeed respectant leurs invariants, tandis qu'un objet statique possède une Hitbox respectant son invariant.

Pré-conditions

prop_pre_moveObject :: Object -> ScreenScrollingSpeed -> Bool

> vérifie que la ScreenScrollingSpeed donnée est positive.

Post-conditions

prop_post_moveObject :: ObjectSpeed -> Bool

> vérifie que les attributs de l'objet, hors sa Hitbox, sont préservés après moveObject.

Propriétés algébriques

prop_commutativity_collisionObject :: Object -> Object -> Bool

> vérifie la commutativité de l'opération collisionObject

Graphics/Assets.hs

Lois

```
law_renderable_finite :: Renderable a => GameAssets -> a -> Bool
```

> doit être vérifié par une instance de `Renderable`, assure que l'entité représentée par une `[Picture]` le soit pour une liste de taille raisonnable (moins de 100 `Picture`).

Pré-conditions

```
prop_pre_completeScoreString :: String -> Bool
```

> vérifie qu'une `String` passée à `completeScoreString` n'est faite que de maximum 7 chiffres.

```
prop_pre_scoreToString :: Score -> Bool
```

> vérifie qu'un `Score` passé à `scoreToString` est positif.

```
prop_pre_getTranslatedScoreAssets :: GameAssets -> Bool -> Score -> Bool
```

> vérifie qu'un `Score` passé à `scoreToString` est positif.

```
prop_pre_getHealthAsset :: GameAssets -> Health -> Bool
```

> vérifie que la `Health` passée à `getHealthAsset` est dans `]0, 100]`.

```
prop_pre_getTranslatedHealthAssets :: GameAssets -> Health -> Health -> Bool
```

> vérifie que les deux `Health` passées à `getTranslatedHealthAssets` sont dans `[0, 100]`.

```
prop_pre_getDigitAsset :: GameAssets -> Int -> Bool
```

> vérifie que l'entier est un unique chiffre (donc dans `[0, 9]`).

Post-conditions

```
prop_post_completeScoreString :: String -> Bool
```

> vérifie que la `String` obtenue par `completeScoreString` contient exactement 7 chiffres.

```
prop_post_scoreToString :: Score -> Bool
```

> vérifie que la `String` obtenue par `scoreToString` contient exactement 7 chiffres, bornée par "9999999".

```
prop_post_getTranslatedScoreAssets :: GameAssets -> Bool -> Score -> Bool
```

> vérifie que la liste `[Picture]` en retour de `getTranslatedScoreAssets` est exactement de taille 7.

```
prop_post_getTranslatedHealthAssets :: GameAssets -> Health -> Health -> Bool
```

> vérifie que le nombre d'assets retourné par `getTranslatedHealthAssets` correspond aux vies données des joueurs (pas d'asset pour un joueur ayant 0 de vie par exemple).

Graphics/Background.hs

Invariants

```
prop_inv_background :: Background -> Bool
```

> spécifie qu'un `Background` doit être placé avec une ordonnée Y dans `[0, heightBackgroundPicture]`.

Pré-conditions

`prop_pre_updateBackground :: Float -> Background -> Bool`

> vérifie que le flottant donné est positif.

Post-conditions

`prop_post_updateBackground :: Float -> Background -> Bool`

> vérifie que les informations du `Background` restent inchangées, sauf son `Y`, après un `updateBackground`.

`prop_post_getTranslatedBackgrounds :: Background -> Bool`

> vérifie qu'une liste d'exactly 3 `Picture` (arrière-plans se suivants) est récupérée par `getTranslatedBackgrounds`.

Graphics/Explosion.hs

Invariants

`prop_inv_explosion :: Explosion -> Bool`

> vérifie que le compteur interne de frames de l'explosion est dans `[1, nbFramesPerExplosionPhase]`, et que le numéro de la phase d'explosion est dans `[0, nbHitAssets-1]`.

Pré-conditions

`prop_pre_getExplosions :: [a] -> [a] -> Bool`

> vérifie que la 2ème liste, supposée être après des collisions, est de taille égale ou plus petite à la première, supposée avant des collisions.

Post-conditions

`prop_post_runExplosion :: Explosion -> Bool`

> vérifie le déroulement correct de la mise à jour de l'explosion (donc l'animation), pour `runExplosion`.

`prop_post_getExplosions :: (Collidable a, Eq a) => [a] -> [a] -> Bool`

> vérifie que si les 2 listes données sont identiques, aucune explosion n'est créée par `getExplosions`.

GameState/Rock.hs

Invariants

`prop_inv_rock :: Rock -> Bool`

> vérifie que le `Rock` possède un `Object` statique, et que l'asset est dans `[0, (nbRockAssets-1)]`.

Pré-conditions

`prop_pre_moveRock :: Rock -> ScreenScrollingSpeed -> Bool`

> vérifie que la `ScreenScrollingSpeed` strictement est positive.

Post-conditions

prop_post_moveRock :: Rock -> ScreenScrollingSpeed -> Bool

> vérifie que tous les attributs autre que le Object contenu restent identiques après le moveRock, et que le type de Rock n'a pas changé (Left/RightRock).

prop_post_getTranslatedRockAsset :: GameAssets -> Rock -> Bool

> vérifie que seul un Picture est retourné par getTranslatedRockAsset.

GameState/Wall.hs

Invariants

prop_inv_finiteWall :: (Invariant a, Collidable a) => FiniteWall a -> Bool

> vérifie que pour tous les éléments contenus dans le FiniteWall, ceux-ci vérifient tous leur invariant (via Invariant), et chacun est en collision avec son suivant dans le mur (via Collidable).

prop_inv_infiniteWall :: Invariant a => InfiniteWall a -> Bool

> vérifie pour un préfixe fini du InfiniteWall, que ce préfixe n'est pas vide et que les éléments du préfixe vérifient tous leur invariant (via Invariant).

prop_inv_gameWalls :: GameWalls -> Bool

> vérifie que les invariants des murs contenus sont satisfaits.

Pré-conditions

prop_pre_moveFiniteWall :: FiniteWall a -> ScreenScrollingSpeed -> Bool

> vérifie que la ScreenScrollingSpeed est strictement positive.

prop_pre_partialMoveInfiniteWall :: InfiniteWall a -> ScreenScrollingSpeed -> Bool

> vérifie que la ScreenScrollingSpeed est strictement positive.

prop_pre_moveGameWalls :: GameWalls -> ScreenScrollingSpeed -> Bool

> vérifie que la ScreenScrollingSpeed est strictement positive.

Post-conditions

prop_post_filterWall :: (a -> Bool) -> FiniteWall a -> Bool

> vérifie que le FiniteWall résultant de filterWall a une taille inférieure ou égale à avant le filtrage.

prop_post_moveFiniteWall :: (Movable a) => FiniteWall a -> ScreenScrollingSpeed -> Bool

> vérifie que la longueur du FiniteWall reste la même après le moveFiniteWall.

prop_post_infiniteWallToFiniteList :: InfiniteWall a -> Bool

> vérifie que la longueur de la liste obtenue par infiniteWallToFiniteList est nécessairement "nbTakeInfiniteWalls".

prop_post_partialMoveInfiniteWall :: (Eq a, Movable a) => InfiniteWall a -> ScreenScrollingSpeed -> Bool

> vérifie que le résultat de partialMoveInfiniteWall sur l'InfiniteWall en paramètre est égal au fait d'appliquer move sur le

préfixe du `InfiniteWall` donné en paramètre.

```
prop_post_addFiniteWall :: GameWalls -> FiniteWall Rock -> Bool
```

> vérifie que le `GameWalls` n'a pas vu ses murs infinis modifiés après le `addFiniteWall`, et que la liste des murs finis est la même que précédemment, avec donc en plus le `FiniteWall Rock` donné.

Propriétés

```
prop_wall_allCollideWithNext :: Collidable a => [a] -> Bool
```

> vérifie que chaque élément de la liste est en collision avec son suivant.

GameState/Projectile.hs

Invariants

```
prop_inv_projectile :: Projectile -> Bool
```

> vérifie, à la fois pour un `PlayerShot` et un `EnemyShot` que l'invariant sur le `Object` interne est maintenu, que cet `Object` se déplace par lui-même (pas un objet statique), que le numéro de l'asset du projectile est correct, que les dégâts sont strictement positifs, et pour le `PlayerShot` que l'id du joueur est 1 ou 2.

Pré-conditions

```
prop_pre_startInitPlayerShot :: XCoord -> YCoord -> ObjectSpeed -> ProjectileAsset -> Damage -> PlayerId -> Bool
```

> vérifie que le `ProjectileAsset`, les `Damage`, et le `PlayerId` sont corrects par rapport à l'invariant.

```
prop_pre_startInitEnemyShot :: XCoord -> YCoord -> ObjectSpeed -> ProjectileAsset -> Damage -> Bool
```

> vérifie que le `ProjectileAsset` et les `Damage` sont corrects par rapport à l'invariant.

```
prop_pre_moveProjectile :: Projectile -> ScreenScrollingSpeed -> Bool
```

> vérifie que la `ScreenScrollingSpeed` est strictement positive.

Post-conditions

```
prop_post_moveProjectile :: Projectile -> ScreenScrollingSpeed -> Bool
```

> vérifie que les attributs autre que le `Object` sont restés inchangés après le `moveProjectile`, tout comme le type de projectile.

```
prop_post_getTranslatedProjectileAsset :: GameAssets -> Projectile -> Bool
```

> vérifie qu'un unique `Picture` est renvoyé par `getTranslatedProjectileAsset`.

GameState/Bonus.hs

Invariants

```
prop_inv_bonus :: Bonus -> Bool
```

> vérifie que l'`Object` donné est statique et vérifie son invariant, ainsi que pour le `PlayerShootBonus`.

```
prop_inv_playerShootBonus :: PlayerShootBonus -> Bool
```

> forcément vrai. Présent juste pour instancier `Invariant`.

Pré-conditions

```
prop_pre_generateNewBonuses :: StdGen -> [Enemy] -> [Enemy] -> Bool
```

> vérifie que la longueur de la 2ème liste d'ennemis est égale ou plus petite que la 1ère.

```
prop_pre_moveBonus :: Bonus -> ScreenScrollingSpeed -> Bool
```

> vérifie que la `ScreenScrollingSpeed` est strictement positive.

Post-conditions

```
prop_post_generateBonusForEnemy :: Enemy -> (StdGen, [Bonus]) -> Bool
```

> vérifie que le générateur de retour de `generateBonusForEnemy` a forcément changé (on a dû y générer un pourcentage pour savoir si on devait créer un bonus ou non, ainsi qu'un type de bonus aléatoire), puis si un bonus a été créé, le nouveau bonus est aux coordonnées de l'ennemi passé en paramètre, et sinon aucun bonus n'a été généré (liste des bonus inchangée).

```
prop_post_generateNewBonuses :: StdGen -> [Enemy] -> [Enemy] -> Bool
```

> vérifie pour le retour de `generateNewBonuses` qu'il y a au plus autant de bonus générés que d'ennemis morts, que chaque bonus est à l'emplacement d'un ennemi mort, et que chaque bonus est valide (pas `NoBonus`).

```
prop_post_moveBonus :: Bonus -> ScreenScrollingSpeed -> Bool
```

> vérifie que le `PlayerShootBonus` est resté identique après le `moveBonus`.

```
prop_post_getTranslatedBonusAsset :: GameAssets -> Bonus -> Bool
```

> vérifie qu'un unique `Picture` est renvoyé par `getTranslatedBonusAsset`.

GameState/Player.hs

Invariants

```
prop_inv_player :: Player -> Bool
```

> vérifie que l'invariant sur l'`Object` est maintenu, que le `PlayerId` est 1 ou 2, que `Lifes` soit dans [1, 3], `Health` soit dans]0, 100], et `PlayerShootBonus` soit valide pour le joueur non mort, que le `Score` soit positif, que les compteurs de frames soient dans les intervalles donnés, et que la phase d'animation d'explosion pour joueur mort soit dans [0, nbPlayerExplosionAssets].

Pré-conditions

```
prop_pre_startInitPlayer :: PlayerId -> Bool
```

> vérifie avant d'appeler `startInitAlivePlayer` ou `startInitDeadPlayer` que le `PlayerId` entier est 1 ou 2.

```
prop_pre_aliveToInvinciblePlayer :: Player -> Bool
```

> vérifie qu'avant de passer invincible, le joueur vivant a plus d'une `Lifes` et a exactement une `Health` de 0.

```
prop_pre_invincibleToAlivePlayer :: Player -> Bool
```

> vérifie que le joueur en paramètre est invincible.

prop_pre_addScore :: Score -> Player -> Bool

> vérifie que le Score en paramètre est positif.

prop_pre_movePlayer :: Player -> ScreenScrollingSpeed -> Bool

> vérifie que la ScreenScrollingSpeed est strictement positive.

Post-conditions

prop_post_aliveToInvinciblePlayer :: Player -> Bool

> vérifie que le joueur invincible nouvellement créé par aliveToInvinciblePlayer a tous ses attributs identiques à avant, excepté le fait d'avoir désormais une Lives en moins, une Health réinitialisée à 100, et un compteur de frames d'invincibilité initialisé à 1.

prop_post_invincibleToAlivePlayer :: Player -> Bool

> vérifie que le joueur vivant (normal) nouvellement créé par invincibleToAlivePlayer a tous ses attributs identiques à avant, avec un compteur de frames rouges initialisé à 0 (= joueur non affiché en rouge).

prop_post_updatePlayerObject :: Player -> Object -> Bool

> vérifie que tous les attributs sont les mêmes par rapport à avant, sauf pour le nouvel Object désormais celui du joueur.

prop_post_addScore :: Score -> Player -> Bool

> vérifie que tous les attributs sont les mêmes par rapport à avant, avec le Score donné ajouté à celui auparavant présent chez le joueur.

prop_post_movePlayer :: Player -> ScreenScrollingSpeed -> Bool

> vérifie que tous les attributs sont les mêmes par rapport à avant, sauf pour l'Object.

prop_post_playerShot :: Player -> Bool

> vérifie que qu'un joueur non mort, dont le compteur de frames du délai de tir a atteint ou dépassé le seuil de frames pour tirer, a pu tirer lors de playerShot, et a alors réinitialisé son compteur. Sinon pas de tir et le joueur est inchangé.

prop_post_runPlayerAnimation :: Player -> Bool

> vérifie l'évolution correcte des compteurs de frames pour les animations, que ce soit pour le rouge du joueur normal, déclenché après avoir reçu précédemment des dégâts, ou pour l'invincibilité de l'invincible. Pour ce dernier cas, si le compteur de frames d'invincibilité a atteint sa limite, on vérifie que le joueur est redevenu normal.

prop_post_updatePlayerShootBonus :: Player -> PlayerShootBonus -> Bool

> vérifie que tous les attributs sont les mêmes par rapport à avant, sauf pour le nouveau PlayerShootBonus désormais celui du joueur.

prop_post_incrementShootFrameCounter :: Player -> Bool

> pour les joueurs vivants, vérifie que tous les attributs sont les mêmes par rapport à avant, sauf le compteur de frames du délai de tir, incrémenté.

prop_post_getTranslatedPlayerAsset :: GameAssets -> Player -> Bool

> vérifie qu'au plus un Picture est renvoyé par getTranslatedPlayerAsset (si mort, aucun).

prop_post_getTranslatedBoosterAssets :: GameAssets -> Player -> Bool

> en fonction de la direction du joueur, vérifie qu'au plus 3 Pictures de boosters sont renvoyés par getTranslatedBoosterAssets.

prop_post_takeDamagePlayer :: Damage -> Player -> Bool

> vérifie l'évolution correcte du joueur sur réception de dégâts : si vivant, on vérifie que sur la perte de points de vie, sans perte de vie, celui-ci reste vivant (avec de la vie en moins, le reste est identique). De manière semblable, si il prend plus de dégâts qu'il a de points de vie, on vérifie qu'il passe bien invincible si plus d'une vie restante, et sinon qu'il passe bien mort.

GameState/Enemy.hs

Invariants

prop_inv_enemy :: Enemy -> Bool

> vérifie que l'invariant sur l'Object est maintenu, que la Health, les Damage, et les Score donné soient strictement positifs, et que le EnemyScript vérifie son invariant.

prop_inv_enemyScript :: EnemyScript -> Bool

> vérifie que les attributs des scripts d'ennemis ont des valeurs correctes dans les intervalles donnés.

Pré-conditions

prop_pre_moveEnemy :: Enemy -> ScreenScrollingSpeed -> Bool

> vérifie que la ScreenScrollingSpeed est strictement positive et que selon l'EnemyScript contenu, l'ennemi possède une direction et une vitesse cohérentes avec son état actuel dans le script.

prop_pre_generateListEnemies :: Int -> StdGen -> Bool

> vérifie que l'entier donné en paramètre, correspondant au nombre d'Enemy à générer, ne dépasse pas 5.

prop_pre_generateFormation :: Int -> Int -> XCoord -> YCoord -> XCoord -> YCoord -> Bool

> vérifie que le type d'Enemy à générer est dans [0, (nbEnemiesAssets-1)], que le nombre est dans [1, 5], et que les coordonnées du centre de la formation permettent d'avoir tous les ennemis, générés par la suite, dans l'écran.

Post-conditions

prop_post_moveEnemy :: Enemy -> ScreenScrollingSpeed -> Bool

> vérifie que les ennemis ont bien bougé comme espéré selon l'état de leur EnemyScript contenu, que les attributs autres que l'Object n'ont pas changé, et que le type de script n'a pas changé.

prop_post_shootEnemy :: Enemy -> Bool

> vérifie que, si le délai le permettait, le LeftRightShootEnemy ait tiré et ait vu son délai de tir être réinitialisé. Les autres attributs sont aussi vérifiés comme ne changeant pas, tout comme le type du script contenu.

prop_post_generateListEnemies :: Int -> StdGen -> Bool

> vérifie qu'en cas de générations d'ennemis, ceux-ci ont tous été générés dans l'écran ou au-dessus.

prop_post_generateFormation :: Int -> Int -> XCoord-> YCoord-> XCoord-> YCoord-> Bool

> vérifie que des ennemis ont été générés, tous dans l'écran ou au-dessus.

prop_post_getTranslatedEnemyAsset :: GameAssets -> Enemy -> Bool

> vérifie qu'un unique Picture est renvoyé par getTranslatedEnemyAsset.

prop_post_takeDamageMaybeEnemy :: Damage -> Enemy -> Bool

> vérifie que si la réception des dégâts mène l'Enemy à une Health<=0, celui-ci alors meurt, et sinon voit seulement sa Health être réduite, le reste devant rester identique.

GameState/Game.hs

Invariants

prop_inv_game :: Game -> Bool

> vérifie que les invariants sur GameState et Background sont maintenus, en plus d'avoir un compteur de frames dans [0, maxFramesToConsider] (reset quand le max est atteint, pour éviter un éventuel overflow).

prop_inv_gameState :: GameState -> Bool

> vérifie essentiellement que l'invariant de InGameInfos est maintenu, dans le cas d'un GameState construit avec le constructeur InGame.

prop_inv_ingameinfos :: InGameInfos -> Bool

> vérifie que la ScreenScrollingSpeed du jeu est strictement positive, et que les invariants sur les 2 Player, la liste d'Enemy, les GameWalls, la liste de Projectile, la liste d'Explosion, et la liste de Bonus vérifient leurs invariants.

Pré-conditions

prop_pre_startInitInGame :: StdGen -> Int -> Bool

> vérifie que le nombre de joueurs donné est 1 ou 2.

prop_pre_updatePlayerDirectionSpeed :: Bool -> (Direction, ObjectSpeed) -> InGameInfos -> Bool

> vérifie que le joueur concerné par updatePlayerDirectionSpeed n'est pas mort.

prop_pre_movePlayerInsideScreen :: Bool -> InGameInfos -> Bool

> vérifie que le joueur concerné par movePlayerInsideScreen n'est pas mort.

prop_pre_handleCollisionPlayerWithEnemies :: Bool -> InGameInfos -> Bool

> vérifie que le joueur concerné par handleCollisionPlayerWithEnemies n'est pas mort.

prop_pre_keepBumpin :: ScreenScrollingSpeed -> Object -> GameWalls -> Player -> Bool

> vérifie que la ScreenScrollingSpeed est strictement positive.

prop_pre_bumpPlayerFromWalls :: Bool -> InGameInfos -> Bool

> vérifie que le joueur concerné par bumpPlayerFromWalls n'est pas mort et n'est pas en collision avec un mur du jeu.

prop_pre_moveProjectiles :: InGameInfos -> Bool

> vérifie que tous les projectiles sont dans l'écran.

prop_pre_applyProjectileToEnemy :: Projectile -> (Maybe Enemy, Score, Score) -> Bool

> vérifie que les scores accumulés des 2 joueurs sont positifs, et que le Projectile en paramètre appartient à un joueur.

prop_pre_moveBonuses :: InGameInfos -> Bool

> vérifie que tous les Bonus du jeu sont dans l'écran ou au-dessus.

prop_pre_handleCollisionPlayerWithBonuses :: Bool -> InGameInfos -> Bool

> vérifie que le joueur concerné par handleCollisionPlayerWithBonuses n'est pas mort.

Post-conditions

<code>prop_post_updatePlayerDirectionSpeed :: Bool -> (Direction, ObjectSpeed) -> InGameInfos -> Bool</code> > vérifie que toutes les infos du jeu restent les mêmes, sauf pour l'<code>Object</code> du joueur concerné. La nouvelle direction et vitesse de l'objet du joueur concerné mises à jour sont vérifiées. De même, on vérifie que le joueur est toujours en vis après le <code>updatePlayerDirectionSpeed</code>.
<code>prop_post_movePlayerInsideScreen :: Bool -> InGameInfos -> Bool</code> > vérifie que toutes les infos du jeu restent les mêmes, sauf pour l'<code>Object</code> du joueur car celui-ci a bougé. De même, on vérifie que le joueur est toujours en vie après <code>movePlayerInsideScreen</code>.
<code>prop_post_handleCollisionPlayerWithEnemies :: Bool -> InGameInfos -> Bool</code> > vérifie que toutes les infos du jeu restent les mêmes, sauf pour la longueur de la liste des ennemis pouvant avoir été réduite, tout comme le nombre de vies du joueur concerné, ou encore on vérifie aussi une possible augmentation du score de ce même joueur.
<code>prop_post_moveWalls :: InGameInfos -> Bool</code> > vérifie que tous les murs du jeu sont dans l'écran, et que les autres infos du jeu n'ont pas changé.
<code>prop_post_keepBumpin :: ScreenScrollingSpeed -> Object -> GameWalls -> Player -> Bool</code> > vérifie que l'<code>Object</code> résultant n'est ni en collision avec un mur du jeu, ni en collision avec l'autre <code>Player</code>, donné en paramètre. En plus de cela, on s'assure que le Y n'a pas changé et que le X de l'objet est dans l'écran.
<code>prop_post_bumpPlayerFromWalls :: Bool -> InGameInfos -> Bool</code> > vérifie que le joueur concerné est toujours vivant, qu'il n'est pas en collision avec un mur du jeu, et que les autres informations du jeu n'ont pas changé.
<code>prop_post_moveProjectiles :: InGameInfos -> Bool</code> > vérifie que les projectiles bougés sont tous dans l'écran, et que les autres informations du jeu n'ont pas changé.
<code>prop_post_applyProjectileToEnemy :: Projectile -> (Maybe Enemy, Score, Score) -> Bool</code> > vérifie qu'au plus un des deux accumulateurs de score a pu être incrémenté, pas les deux en même temps.
<code>prop_post_handleCollisionProjectilesWithPlayersEnemies :: StdGen -> InGameInfos -> Bool</code> > vérifie le fait de pouvoir avoir moins d'ennemis, d'avoir une ou plusieurs vies en moins pour les joueurs, de pouvoir avoir des score de joueurs augmentés, d'avoir potentiellement de nouveaux bonus ou explosions dans l'écran du jeu, et on n'oublie pas de vérifier que le reste est inchangé.
<code>prop_post_runEnemies :: InGameInfos -> Bool</code> > vérifie le fait d'avoir vu bougé les ennemis ayant la capacité de bouger, que le nombre total d'ennemis n'a pas changé (on ne teste pas l'égalité entre la liste originale d'ennemis et celle récupérée car les ennemis ayant bougé n'ont plus la même valeur), on vérifie le fait d'avoir potentiellement plus de projectiles (des ennemis ayant tirés), et le reste des informations doit être identique à avant.
<code>prop_post_runExplosions :: InGameInfos -> Bool</code> > vérifie le fait d'avoir peut être moins d'explosions dans le jeu (celles se terminant étant supprimées), le reste devant être identique.
<code>prop_post_runPlayersAnimation :: InGameInfos -> Bool</code> > vérifie le fait d'avoir toutes les informations identiques à avant, sauf pour les joueurs.
<code>prop_post_generateWall :: FrameCounter -> StdGen -> InGameInfos -> Bool</code> > si pas de génération de nouveaux murs (le compteur n'a pas atteint <code>currentFrameCounter `mod` generateWallInterval == 0</code>, excluant le cas <code>currentFrameCounter==0</code>), alors toutes les informations doivent ne pas avoir changé. Sinon, on vérifie qu'il peut y avoir plus de murs finis, le reste devant être identique à avant.

```
prop_post_moveBonuses :: InGameInfos -> Bool
```

> vérifie que les bonus gardés après le mouvement sont dans l'écran, les autres informations doivent ne pas avoir changé.

```
prop_post_handleCollisionPlayerWithBonuses :: Bool -> InGameInfos -> Bool
```

> vérifie, sauf pour les joueurs ayant peut-être récupéré un bonus, que les autres informations n'ont pas changé par rapport à avant.

```
prop_post_incrementPlayersShootFrameCounters :: InGameInfos -> Bool
```

> vérifie, sauf pour les joueurs ayant vu leur compteur interne de frame de délai de tir incrémenté, que les autres informations n'ont pas changé par rapport à avant.

Description des tests

Concernant mes tests, j'ai un total de **407** tests (unitaires + QuickCheck), ce qui fait que je ne vais pas les présenter de manière individuelle, mais plutôt de manière générale. Je vais néanmoins reprendre une partie de ce que j'ai fait pour illustrer certains points.

Mon **Spec.hs** délègue les tests à tous mes autres fichiers présents dans le répertoire de **test**. Pour chaque fichier source contenant des types ou des opérations, un fichier de test correspondant existe, par exemple le contenu de **src/Objects/Hitbox.hs** est testé dans **test/HitboxSpec.hs**.

Méthodologie générale des tests

Générateurs et constructeurs

Un premier point important concerne la **manière dont je crée les générateurs** de mes données testées. J'avais le choix entre implémenter directement mes types de données comme des instances de **Arbitrary**, et alors devoir le faire dans les fichiers source, mélangeant alors cette logique de test avec le reste, ou de faire des wrappers **Arbitrary** de mes types de données dans mes fichiers de tests. J'ai opté pour la 2ème méthode, ainsi pour chacun de mes types de données, je crée un **wrapper instanciant Arbitrary**, et offrant un accesseur à la donnée représentée. Cela donne par exemple :

```
-- mon wrapper de Hitbox
newtype TestHitbox = TestHitbox { getHitbox :: Hitbox } deriving (Eq, Show)
instance Arbitrary TestHitbox where
  arbitrary :: Gen TestHitbox
  arbitrary = oneof
    [ TestHitbox <$> genCircle
    , TestHitbox <$> genRectangle
    , TestHitbox <$> genHitboxes]
```

pour mon *wrapper* de mon type de données **Hitbox**.

Pour mes implémentations de la fonction **arbitrary**, les générateurs utilisés sont soit définis à l'extérieur de la fonction, la plupart du temps pour des générateurs non triviaux dans le cas où je peux avoir plusieurs constructeurs pour la donnée encapsulée, comme ici avec **Hitbox**, soit directement dans le corps, si un seul constructeur pour la donnée encapsulée est possible ou si les constructeurs sont triviaux, comme pour **Rock** :

```
newtype TestRock = TestRock { getRock :: Rock } deriving (Eq, Show)
instance Arbitrary TestRock where
  arbitrary :: Gen TestRock
  arbitrary = do
    asset <- choose (0, nbRockAssets - 1)
    forward <- arbitrary
```

```

leftSide <- arbitrary
x <- arbitrary
y <- arbitrary

let obj = initStaticObject ((Seq.index rockHitbox asset) x y leftSide)
    rock = if leftSide
        then LeftRock obj asset forward
        else RightRock obj asset forward

return (TestRock rock)

```

Dans tous les cas, je fais en sorte que mes **générateurs génèrent toujours des données contenues respectant leur invariant**.

De plus, si un de mes fichiers de test dépend d'un autre type de donnée, j'utilise alors l'instance du wrapper correspondant pour générer aléatoirement la donnée (qui donc respecte toujours son invariant), celle-ci étant accédée par l'accesseur. Cela peut se faire lors de la nécessité de récupérer une instance aléatoire pour un argument d'un test QuickCheck, ou au sein d'un générateur, comme dans celui de [Projectile](#) :

```

instance Arbitrary TestProjectile where
    arbitrary :: Gen TestProjectile
    arbitrary = do
        (TestObject obj) <- arbitrary -- Object généré, encapsulé et récupéré

        assetPlayerShot <- choose (0, nbPlayerShotAssets - 1)
        playerId <- choose (1, 2)
        assetEnemyShot <- choose (0, nbEnemyShotAssets - 1)
        dmg <- getPositive <$> arbitrary
        shot <- oneof
            [ return (PlayerShot obj assetPlayerShot dmg playerId)
            , return (EnemyShot obj assetEnemyShot dmg)
            ]
        return (TestProjectile shot)

```

Concernant mes tests, les **constructeurs sont testés avec QuickCheck** en leur passant en paramètres des valeurs respectant l'invariant de la donnée. Je crée pour cela des propriétés locales à mes fichiers de test, suffixées par **_preservesInvariant**. Par exemple pour tester les 2 constructeurs de [Object](#), j'ai cela :

```

prop_initMovableObject_preservesInvariant :: TestHitbox -> TestDirection ->
TestObjectSpeed -> Property
prop_initMovableObject_preservesInvariant h d s =
    property $ prop_inv_object (initMovableObject (getHitbox h) (getDirection d)
(getObjectSpeed s))

```

```
prop_initStaticObject_preservesInvariant :: TestHitbox -> Property
prop_initStaticObject_preservesInvariant h =
  property $ prop_inv_object (initStaticObject (getHitbox h))
```

Et sont exécutés ainsi :

```
initObjectSpec :: SpecWith ()
initObjectSpec = do
  describe "initObject (QuickCheck)" $ do
    it "preserves the Movable Object invariant for valid Movable Objects" $
      property prop_initMovableObject_preservesInvariant

    it "preserves the Static Object invariant for valid Static Objects" $
      property prop_initStaticObject_preservesInvariant
```

À noter que si un des paramètres a une valeur particulièrement restrictive, par exemple un entier dans [1, 2], alors je ne prend pas l'entier en paramètre de la propriété, mais le crée directement dans celle-ci. Par exemple, je dois avoir un indice d'asset dans [0, 1] et le numéro du joueur dans [1, 2] pour créer le [Projectile](#) d'un joueur. Cela donne :

```
prop_initPlayerShot_preservesInvariant :: TestObject -> Damage -> Property
prop_initPlayerShot_preservesInvariant (TestObject obj) d =
  -- (nbPlayerShotAssets-1) == 1
  forAll (choose (0, (nbPlayerShotAssets-1))) $ \asset ->
  forAll (choose (1, 2)) $ \pId ->
    d >= 1 ==> prop_inv_projectile (initPlayerShot obj asset d pId)
```

Les paramètres restrictifs générés pour mes tests QuickCheck sur les opérations suivent la même logique.

Test unitaires et QuickCheck

Concernant mes **tests en eux-mêmes**, de manière générale j'essaye de **mélanger tests unitaires et tests QuickCheck** : j'effectue des **tests unitaires notamment sur des cas simples**, mais permettant de vérifier que l'opération retourne le résultat attendu. Ce qu'ils testent est directement documenté dans le `describe ... it` du test unitaire. Par exemple pour les tests unitaires sur l'opération `partOfHitbox` pour une [Hitbox](#) :

```
partOfHitboxSpec :: SpecWith ()
partOfHitboxSpec = do
  describe "partOfHitbox (unit tests)" $ do
    it "(1, 1.8) is part of (Rectangle 0 0 2 2)" $ do
      let r = Rectangle 0 0 2 2
      partOfHitbox 1 1.8 r `shouldBe` True

    it "(-3, 1.8) is not part of (Rectangle 0 0 2 2)" $ do
      let r = Rectangle 0 0 2 2
      partOfHitbox (-3) 1.8 r `shouldBe` False
    ...
```

Si ces tests unitaires ne sont pas effectués sur une opération triviale, et/ou ne sont pas assez représentatifs des scénarios possibles, alors j'y ajoute un test QuickCheck, par exemple pour l'opération `moveBonus` pour le type [Bonus](#) :

```
moveBonusSpec :: Spec
moveBonusSpec = do
  describe "moveBonus (unit tests)" $ do
    it "moves bonus towards bottom according to a screen scrolling speed of 10" $ do
      let b = startInitPlayerShootBonus 0 0 ShootFaster
          b' = moveBonus b 10
          obj = bonusObject b'
      centerHitbox (objectHitbox obj) `shouldBe` (0,(-10))
    it "moves bonus towards bottom according to a screen scrolling speed of 1" $ do
      let b = startInitPlayerShootBonus 0 0 ShootFaster
          b' = moveBonus b 1
          obj = bonusObject b'
      centerHitbox (objectHitbox obj) `shouldBe` (0,(-1))

moveBonusQuickCheckSpec :: Spec
moveBonusQuickCheckSpec = do
  describe "moveBonus (QuickCheck)" $ do
    it "satisfies moveBonus post-condition for valid parameters" $
      property (\(TestBonus b) ss ->
```



```

prop_inv_bonus b && (prop_pre_moveBonus b ss)
==> let b' = moveBonus b ss
      in prop_inv_bonus b' && prop_post_moveBonus b ss
)

```

De manière systématique pour ces tests QuickCheck :

- Les invariants sur les données prises en paramètres de l'opération sont vérifiés.
- L'éventuelle pré-condition de l'opération est vérifiée, pour les paramètres pris en entrée.
- L'éventuelle donnée en sortie est **toujours** vérifiée comme satisfaisant son invariant post-opération.
- L'éventuelle post-condition de l'opération est vérifiée, pour les paramètres originaux pris en entrée par l'opération.

Cette manière de faire pour le dernier point tient surtout au fait que la post-condition, par elle même, effectue l'opération pour les paramètres qui lui sont donnés, par exemple ici le code de `prop_post_moveBonus` est :

```

prop_post_moveBonus :: Bonus -> ScreenScrollingSpeed -> Bool
prop_post_moveBonus b@(PlayerBonus _ psb) ss =
  -- le résultat de l'opération est calculé au sein de la post-condition même
  let (PlayerBonus _ psb') = moveBonus b ss
  in psb == psb'

```

À noter que **moins les opérations sont triviales** et plus elles nécessitent de données à initialiser, et plus j'ai **tendance à directement faire des test QuickCheck** : ceux-ci couvrent dans tous les cas BEAUCOUP plus que mes tests unitaires, et si mes propriétés sont bonnes (ce qui est normalement le cas), alors les résultats seront forcément également corrects et permettront d'attester de la validité du comportement de l'opération testée.

De plus, lorsque j'ai des propriétés à tester (par exemple des **propriétés algébriques**), **j'utilise directement QuickCheck**.

Vérification des lois

Une partie importante de mon projet correspond à l'utilisation de plusieurs *typeclasses* créées pour l'occasion. Je reviendrai dans la suite sur cela, mais concernant les tests je me suis donc fixé comme contrainte de **vérifier avec QuickCheck l'exactitude de ces lois** pour mes instances. Ces vérifications s'effectuent en général à la fin de mes fichiers de tests, avec par exemple pour la vérification de la *typeclass* `Collidable` instancié par `Rock` :

```
collidableLawsSpec :: Spec
collidableLawsSpec = do
  describe "Collidable laws (QuickCheck)" $ do
    it "law_collidable_reflexive for Rock" $
      property ( \(TestRock o) ->
        prop_inv_rock o
        ==> law_collidable_reflexive o
      )
    it "law_collidable_symmetric for Rock with another Rock" $
      property ( \(TestRock r1) (TestRock r2) ->
        prop_inv_rock r1 && prop_inv_rock r2
        ==> law_collidable_symmetric r1 r2
      )
    it "law_collidable_symmetric for Rock with another Object" $
      property ( \(TestRock r1) (TestObject o2) ->
        prop_inv_rock r1 && prop_inv_object o2
        ==> law_collidable_symmetric r1 o2
      )

    it "law_collidable_will_collide for Rock with another Rock" $
      property ( \(TestRock r1) (TestRock r2) ->
        prop_inv_rock r1 && prop_inv_rock r2
        ==> law_collidable_will_collide r1 r2
      )
    it "law_collidable_will_collide for Rock with another Object" $
      property ( \(TestRock r1) (TestObject o2) ->
        prop_inv_rock r1 && prop_inv_object o2
        ==> law_collidable_will_collide r1 o2
      )
```

Pour la cas spécifique à `Collidable`, elle nécessite d'avoir un autre élément `Collidable` en 2ème paramètre à certaines de ses opérations / lois. Ce 2ème paramètre est systématiquement testé d'abord comme étant du **même type de donnée**, puis du **type Object** (c'est ce type qui dans les faits est utilisé par les instances pour gérer chaque mouvement, c'est donc le type externe le plus pertinent à utiliser, et je n'allais pas faire ces vérifications avec TOUTES mes autres instance de `Collidable`, moins pertinent + il y en a trop).

Point pertinent : génération de Hitboxes

Pour mon type `Hitbox`, je possède un constructeur : `Hitboxes XCenter YCenter [Hitbox]`

Viennent 2 challenges : s'assurer que le `(XCenter,YCenter)` fait bien partie de la liste de `Hitbox` contenue, et que l'on ne se retrouve pas dans une situation où une génération aléatoire pourrait mener de manière réursive à une `Hitboxes` ayant dans sa liste une autre `Hitboxes`, qui-elle même a dans sa liste une autre `Hitboxes`, etc...

Pour pallier à cela, je me suis fait ce générateur :

```
genHitboxes :: Gen Hitbox
genHitboxes = do
  x <- arbitrary
  y <- arbitrary

  base <- oneof -- 1er élément de la liste de Hitbox
    [ Circle x y . abs <$> arbitrary
    , pure (Rectangle (x - 1) (y - 1) 2 2) ]

  n <- choose (0, 9)
  let genAtomic = oneof
    [ TestHitbox <$> genCircle
    , TestHitbox <$> genRectangle ]

  rest <- vectorOf n genAtomic
  return (Hitboxes x y (base : (map getHitbox rest)))
```

L'idée est que je génère une liste interne de `Hitbox` ayant une taille totale entre 1 et 10 éléments, en fixant le premier élément comme contenant le centre de mon `Hitboxes` généré.

S'ajoutent ensuite les éléments suivants de la liste, utilisant nécessairement les constructeurs `Circle` ou `Rectangle`, en utilisant les générateurs faits pour ces constructeurs.

Ainsi, la liste est de longueur finie et évite d'avoir un cas de récursion infinie de listes contenues, tout en gardant une dimension arbitraire par la taille raisonnable de la liste et ses éléments contenus.

Dans d'autres fichiers de tests, je me retrouve à devoir procéder de la même manière pour gérer la création aléatoire de listes d'éléments, en évitant ce genre de récursions et en fixant une borne maximale sur la taille de la liste générée.

Point pertinent : génération des murs finis et infinis

D'abord pour mes **murs finis**, de type (`FiniteWall a`), ces murs doivent être non vides, et surtout doivent avoir leurs éléments se chevauchant, les uns à la suite des autres. Étant donné le fait d'avoir un type paramétré, afin de concrètement tester ce type je dois commencer par fixer ce 'a', représenté alors par le type `Rock` tel que pour mon utilisation dans le cadre de mon jeu.

`Rock` étant instance des *typeclasses* `Collidable` et `Invariant`, cela étant nécessaire pour maintenir l'invariant sur mon `FiniteWall`, alors il ne me reste plus qu'à présenter le générateur :

```
newtype TestFiniteWall a = TestFiniteWall { getFiniteWall :: FiniteWall a }
deriving (Eq, Show)
instance Arbitrary (TestFiniteWall Rock) where
  arbitrary :: Gen (TestFiniteWall Rock)
  arbitrary = do
    n <- choose(1,200)

    -- la position du 1er Rock est générée aléatoirement
    x0 <- arbitrary
    y0 <- arbitrary
    objs <- mkRocks n (x0,y0) [] -- appel à une fonction auxiliaire
    return $ TestFiniteWall (initFiniteWall objs)

  where
    -- Génère un mur de Rock se chevauchant
    -- 1er paramètre : le nombre de Rock restants à créer
    -- 2ème paramètre : (x, y) du centre du Rock à créer couramment
    -- 3ème paramètre : la liste contenant tous les Rocks du mur, jusqu'à
    présent générés
    mkRocks :: Int -> (Float, Float) -> [Rock] -> Gen [Rock]
    mkRocks 0 _ acc = return (reverse acc)
    mkRocks k (x,y) acc = do
      leftSide <- arbitrary
      forward <- arbitrary
      asset <- choose(0, (nbRockAssets-1))

      let obj = initStaticObject ((Seq.index rockHitbox asset) x y
leftSide)

      rock = if leftSide
              then LeftRock obj asset forward
              else RightRock obj asset forward

      xNext = x
      yNext = y + 15 -- vertical overlap for rocks case
```

```
mkRocks (k-1) (xNext,yNext) (rock : acc)
```

Comme précédemment, la taille de la liste est fixée de manière raisonnable, le premier **Rock** est généré avec une position aléatoire, et ensuite la fonction auxiliaire permet de créer une liste verticale de **Rock** se chevauchant. Ce chevauchement étant assuré par une différence d'abscisse de 15 pixels entre les centres de chaque Rock : pour vous donner une idée, selon l'asset du **Rock** celui-ci a une hauteur variant de 42 à 44 pixels, donc les chevauchements sont certains.

Ensuite pour les **murs infinis**, voici le générateur :

```
newtype TestInfiniteWall a = TestInfiniteWall { getInfiniteWall :: InfiniteWall a
} deriving (Eq, Show)
instance Arbitrary (TestInfiniteWall Rock) where
  arbitrary :: Gen (TestInfiniteWall Rock)
  arbitrary = do
    foreground <- arbitrary
    left <- arbitrary
    seed <- arbitrary  :: Gen Int

    let gen = mkStdGen seed
        -- on appelle directement le constructeur de murs infinis
        -- du code source, dans GameState/Wall.hs
        wall = initInfiniteWall foreground left gen

    return (TestInfiniteWall wall)
```

Avec le constructeur appelé :

```
initInfiniteWall :: Bool -> Bool -> StdGen -> InfiniteWall Rock
initInfiniteWall foreground left gen =
  let
    -- Largeur maximale parmi tous les assets de rochers.
    -- Utilisée pour calculer jusqu'où les murs peuvent dépasser de l'écran.
    maxW = maximum (toList widthRocks)
    -- Dépassement horizontal maximal autorisé hors de l'écran.
    overflow = (maxW/2) * 0.6

    --- Bornes aléatoires de la position X.
    -- Les murs à gauche peuvent légèrement dépasser à gauche,
    -- ceux à droite peuvent légèrement dépasser à droite.
    (lowerX, upperX) =
      if left
      then (leftXScreenBound - overflow, leftXScreenBound + overflow)
      else (rightXScreenBound - overflow, rightXScreenBound + overflow)
```

```

-- Coordonnée Y de départ.
-- Les murs d'arrière-plan sont décalés verticalement d'une demi-case.
baseY = if foreground then bottomYScreenBound
        else bottomYScreenBound+(rockCell/2)

-- Suite infinie de coordonnées Y pour les segments du mur (Rocks).
-- Les nbTakeInfiniteWalls premières valeurs sont espacées normalement,
-- puis toutes les suivantes restent constantes.
ys =
  let firstYs =
    map (\i -> baseY + fromIntegral i * rockCell)
      ([0 .. nbTakeInfiniteWalls - 1] :: [Int])

    lastY = last firstYs
  in firstYs ++ repeat lastY

-- Index aléatoires des assets de rochers
randomWalls = randomRs (0, nbRockAssets - 1) gen
-- Positions X aléatoires des Rocks, pouvant être hors écran
randomX = randomRs (lowerX, upperX) gen

-- Associe chaque index d'asset de Rock à une position X aléatoire
randomValues = zip randomWalls randomX

in
  let wall = InfiniteWall (zipWith makeWallObject ys randomValues)
  in
    if not (prop_inv_infiniteWall wall)
    then error "invalid infinite wall"
    else wall

where
  -- Crée un objet rocher individuel faisant partie du mur infini
  -- Premier argument : position Y
  -- Deuxième argument : paire (index de l'asset Rock, position X)
  makeWallObject :: YCoord -> (Int, XCoord) -> Rock
  makeWallObject y (numRock, x) =
    let x2 = if left then x else

        -- Décalage côté droit si les Rocks ont une largeur < 90
        case numRock of
          0 -> x

```

```

1 -> x
2 -> x+3
3 -> x+6
i -> error (show i++ " out of bounds")
-- startInitRock est un constructeur de Rock, initialisant dans un
-- état de début
in startInitRock x2 y numRock left foreground

```

Ce que j'effectue ici est le fait d'appeler le **même constructeur que celui appelé au début d'une partie** pour initialiser un mur infini : celui-ci est alors également un mur vertical, soit côté gauche, soit côté droit de l'écran, pouvant être en avant ou en arrière plan (tout cela fixé grâce aux paramètres du constructeur).

Ce qui est le plus intéressant est la création des données aléatoires concernant les Rock du mur :

- **Les coordonnées Y sont faites dans une liste infinie :**

```

ys =
  let firstYs =
    map (\i -> baseY + fromIntegral i * rockCell)
      ([0 .. nbTakeInfiniteWalls - 1] :: [Int])

    lastY = last firstYs
  in firstYs ++ repeat lastY

```

Les "nbTakeInfiniteWalls" premiers Y du mur sont incrémentés à chaque itération de 42 pixels (= rockCell), menant jusqu'à un dernier Y de ce préfixe, au-dessus de l'écran de jeu, pour lequel toutes les valeurs infinies suivantes sont constantes.

Etant donné que **l'on ne peut pas faire d'opérations sur un mur infini entier**, l'idée du mouvement dans notre boucle de jeu sera alors de faire **à chaque fois bouger vers le bas le préfixe du mur infini, supprimer les Rock qui passent hors de l'écran, en bas, et continuer ainsi à chaque tour de boucle du jeu** : ainsi les nouvelles Rocks à la fin du préfixe, venant de l'intégrer et donc bougeant vers le bas, vont **commencer à un Y nécessairement au-dessus de l'écran de jeu**.

- **Les index des assets des Rocks générés sont aussi faits dans une liste infinie**
`randomWalls = randomRs (0, nbRockAssets - 1) gen`
- **Les X des Rocks**, générés dans un intervalle leur permettant de déborder un peu hors de l'écran pour empêcher le fait d'avoir un mur de largeur parfaitement homogène, **sont aussi faits dans une liste infinie** :
`randomX = randomRs (lowerX, upperX) gen`

Ces 3 listes sont combinées avec `zip` et `zipWith`, et les éléments individuels de chaque liste précédente permettent à la fonction auxiliaire `makeWallObject` de générer également de manière *lazy* une liste infinie de **Rock**, faisant donc le **mur infini**.

Ainsi, lors de l'exécution du jeu on a cette impression de murs infinis, avec effectivement des valeurs générées dès le début, qui ne sont exécutées qu'en considérant et mettant à jour les préfixes des murs infinis.

Je pense aussi important de préciser que j'ai eu **quelques difficultés à arriver à ce résultat** (dont je suis quand même un peu fier), notamment car j'avais eu la mauvaise idée au départ d'effectuer à chaque tour de boucle du jeu un `fmap move` sur l'ensemble du mur infini : cela ne me faisait pas planter mon jeu, qui pouvait tourner, mais vu que plus on avançait dans le jeu, et plus d'appels de `fmap` se stackaient sur les [Rocks](#) exécutées lors de leur affichage (qui se faisait déjà sur seulement un préfixe du mur infini), alors au bout de 40/50 secondes de jeu, l'exécution devenait vite quasiment impossible car le jeu ralentissait de plus en plus à cause des `fmap` stackés que j'ai évoqué avant.

Description du projet

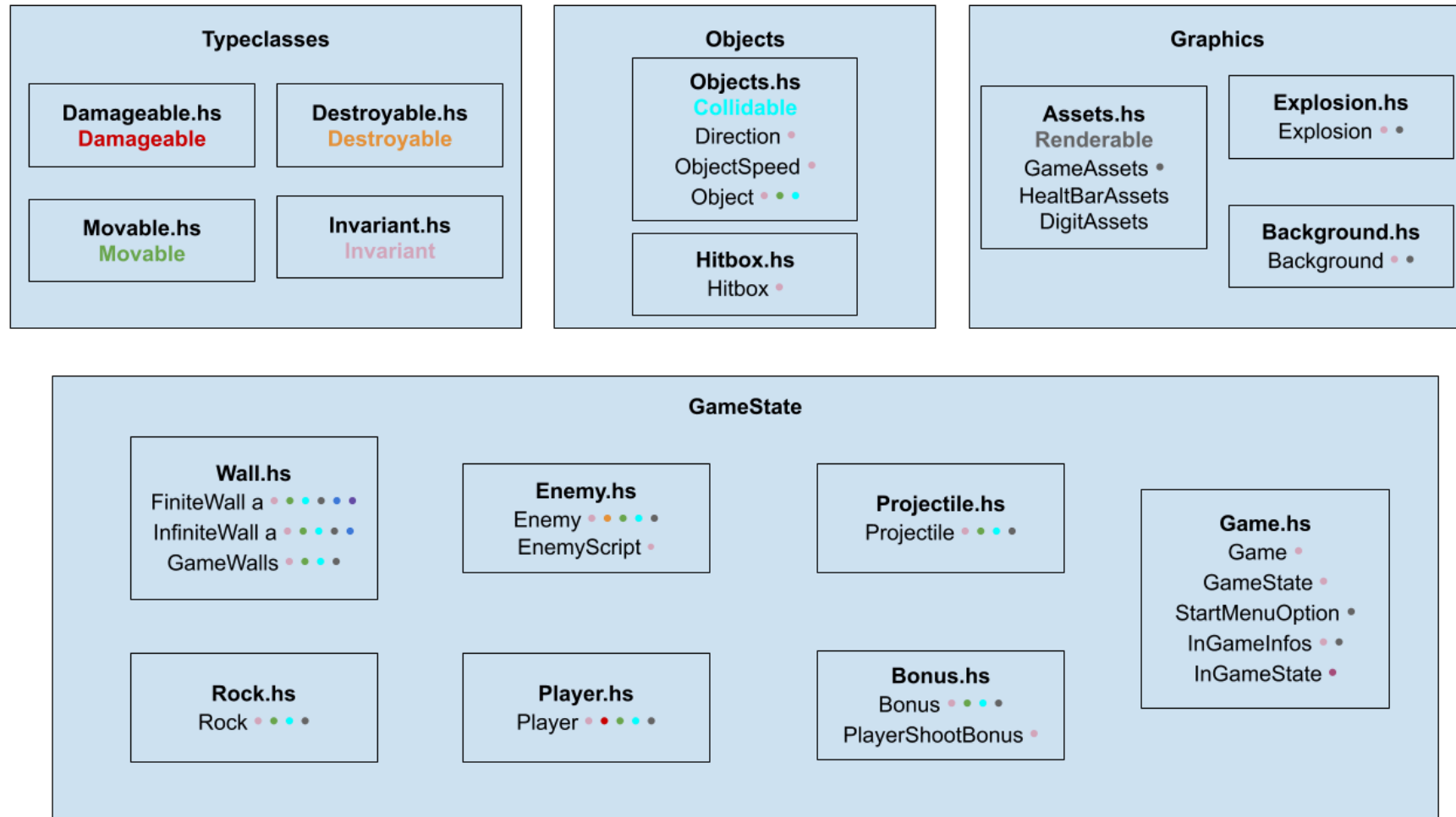
Schéma des classes de types et types de données (à partir du répertoire **src**)

Légende des instanciations :

- **Damageable**
- **Destroyable**
- **Movable**
- **Collidable**
- **Renderable**
- **Invariant**
- **Functor**
- **Foldable**
- **State**

Quasiment tous les types
instancient Eq et Show

Les types alias ne sont pas
représentés



Description globale de l'architecture

Concernant mon fichier **app/Main.hs**, celui-ci ne contient que le minimum de la prise en charge des entrées/sorties, de sorte à pouvoir avoir tous mes traitements dans des fonctions pures. Ainsi j'ai mes 3 fonctions Gloss :

- **renderIO** : s'occupe essentiellement d'afficher soit le menu d'accueil, soit les assets du jeu en lui-même. La **typeclass** **Renderable** est particulièrement utile dans ce cas puisqu'elle me permet en un seul appel, `getTranslatedAssets ga igi`, d'afficher l'ensemble des assets de la partie en cours.
- **handleEventsIO** : s'occupe de prendre en charge les entrées clavier de mes joueurs, pour le menu d'accueil ou en pleine partie.
- **updateIO** : ne faisant rien pour le menu d'accueil, mais mettant à jour l'arrière-plan, l'état de la partie en lui-même par **délégation à une fonction d'update de ma monade State**, et le compteur de frames lorsqu'une partie est en cours.

Concernant ma **gestion de mes types de données**, j'ai particulièrement insisté sur le fait d'**introduire des typeclasses** me permettant d'effectuer des fonctions ayant certaines sémantiques spécifiques, parfois indépendantes des types concrets de données, mais permettant donc de facilement appliquer une opération sur différentes entités. Je vais par la suite les détailler un peu plus en détail, en introduisant également leurs lois.

En terme global de types de données :

- Ceux représentant les **mouvements et positions spatiales** de mes entités en jeu sont dans le répertoire **src/Objects**.
- Ceux ayant uniquement une **utilité graphique** sont dans le répertoire **src/Graphics**.
- Ceux représentant mes **entités en jeu** sont dans le répertoire **src/GameState**.
- Ma **boucle principale** ainsi que toutes les fonctions modifiant mon état en jeu sont dans le fichier **src/GameState/Game**, avec notamment ma monade State à la fin.

Une autre chose importante à savoir concernant mes initialisations est le fait que j'ai **deux types de smart constructors** :

- Ceux préfixés par "init", prenant tous les paramètres permettant de créer la donnée, et en cas de non respect d'un invariant renvoie une erreur.
- Ceux préfixés par "startInit", prenant en général moins de paramètres, puisque ceux-ci vont initialiser la donnée dans son état initial.

Par exemple pour **initialiser un joueur vivant**, j'ai en smart constructor basique :

```
initAlivePlayer :: Object -> PlayerId -> Lifes -> Health -> Score ->
PlayerShootBonus -> FrameCounter -> FrameCounter -> Player
initAlivePlayer po pId lifes health score psb frameShootCpt frameRedCpt
  | lifes < 0 || lifes > 3 = error "number of lifes outside of [0, 3], must be
                                inside it"
  | health < 0 || health > 100 = error "current life health outside of [0, 100],
                                       must be inside it"
  | score < 0 = error "score must be positive"
```

```

| frameShootCpt < 1 = error "frameShootCpt must be at least 1"
| frameRedCpt < 0 || frameRedCpt > nbFramesRedOnDamage = error "frameRedCpt
                        must be inside [0, nbFramesRedOnDamage]"
| otherwise =
    let p = AliveP po pId lifes health score psb frameShootCpt frameRedCpt
    in if not (insideScreenPlayer p) then error "player out of screen"
    else p

```

Et pour l'**initialiser au début d'une partie**, j'ai comme autre smart constructor :

```

startInitAlivePlayer :: PlayerId -> Player
startInitAlivePlayer pId =
    let xP = if pId == 1 then player1StartX else player2StartX
        yP = if pId == 1 then player1StartY else player2StartY
        newPo = initPlayerObject xP yP (initDirection 0 0) (initObjectSpeed 0)
    in initAlivePlayer newPo pId 3 100 0 NoBonus 1 0

```

(avec une pré-condition sur le `PlayerId`, devant être 1 ou 2)

En cas de modification d'une donnée, je la reconstruis TOUJOURS par mon premier type de smart constructor, me garantissant donc toujours que mes données manipulées satisfont mes invariants. Cela veut dire que, sauf exceptions dans certains fichiers de tests, dans mon code source je ne passe JAMAIS par les constructeurs normaux, mais toujours pas mes smart constructeurs safe.

Description des classes de types et types de données

(à partir du répertoire **src**)

Pour voir les opérations sur les données concernées, se référer au code source dans lequel j'ai essayé de mettre le plus de commentaires pertinents possibles.

GameSetup.hs

Ce fichier ne contient que les types alias généralement utilisés dans le projet pour clarifier les arguments, ainsi que des paramètres généraux du jeu : taille de l'écran et des assets, vitesse de l'écran et des entités, paramètres des joueurs et des ennemis, etc...

Keyboard.hs

```
type Keyboard = Set Key
```

> Permet de savoir quelles touches sont couramment appuyées.

Typeclasses/Damageable.hs

```
class Damageable a where
    -- Indique si un élément Damageable est actuellement mort.
    isDead :: a -> Bool
    -- Applique une quantité de dégâts donnée, et peut modifier l'entité en
    -- conséquence.
    takeDamage :: Damage -> a -> a

-- Un Damageable mort reste mort après sa mort
law_damageable_dead_stays_dead :: Damageable a => Damage -> a -> Bool
law_damageable_dead_stays_dead d a =
    let a' = takeDamage d a
    in if isDead a then isDead a' else True

-- Infliger plusieurs dégâts après la mort ne modifie pas l'entité concernée
law_damageable_dead_idempotent :: (Eq a, Damageable a) => Damage -> Damage -> a ->
Bool
law_damageable_dead_idempotent d1 d2 a =
    let a1 = takeDamage d1 a
    in if isDead a1
        then takeDamage d2 a1 == a1
        else True

-- Aucun dégât = identité
law_damageable_zero_damage_identity :: (Eq a, Damageable a) => a -> Bool
law_damageable_zero_damage_identity a = takeDamage 0 a == a

-- Les dégâts négatifs ne soignent pas
law_damageable_no_heal_negative_damage :: (Eq a, Damageable a) => Damage -> a ->
Property
law_damageable_no_heal_negative_damage d a = d < 0 ==> takeDamage d a == a
```

La *typeclass* `Damageable` sert surtout à garantir une cohérence de comportement pour une entité pouvant prendre des dégâts, et encore exister après sa mort.

Seul `Player` l'implémente, et à la base j'avais plutôt une *typeclass* ressemblant à l'actuelle `Destroyable`, mais en termes de cohérence je me suis rendu compte qu'il y avait trop de difficultés à alors avoir cette gestion pour le `Player`, pouvant encore exister après sa mort, et en même temps avoir `Enemy` implantant la même *typeclass*, lui n'existant pas après sa mort. J'ai donc décidé de séparer en 2 *typeclass* distinctes : `Damageable` ici et `Destroyable` vu juste après.

Typeclasses/Destroyable.hs

```
class Destroyable a where
    -- Applique une quantité de dégâts donnée.
    -- L'entité recevant les dégâts peut mourir et donc retourner Nothing.
    takeDamageMaybe :: Damage -> a -> Maybe a

-- Les dégâts sont cumulatifs tant que l'entité est vivante
law_destroyable_damage_cumulative :: (Eq a, Destroyable a) => Damage -> Damage ->
a -> Bool
law_destroyable_damage_cumulative d1 d2 a =
    case (takeDamageMaybe d1 a) of
        Just a' -> (takeDamageMaybe d2 a') == takeDamageMaybe (d1 + d2) a
        Nothing -> True

-- Aucun dégât = identité
law_destroyable_zero_damage_identity :: (Eq a, Destroyable a) => a -> Bool
law_destroyable_zero_damage_identity a = takeDamageMaybe 0 a == Just a

-- Les dégâts négatifs ne soignent pas
law_destroyable_no_heal_negative_damage :: (Eq a, Destroyable a) => Damage -> a ->
Property
law_destroyable_no_heal_negative_damage d a =
    d < 0 ==> takeDamageMaybe 0 a == Just a
```

La *typeclass* `Destroyable` sert surtout à garantir une cohérence de comportement pour une entité pouvant prendre des dégâts, mais n'existant plus après sa mort. Seul `Enemy` l'implémente.

Typeclasses/Movable.hs

```
class Movable a where
    -- Déplace l'entité en fonction soit de la vitesse de défilement
    -- de l'écran donnée, soit de sa propre vitesse
    move :: a -> ScreenScrollingSpeed -> a
    -- Indique si l'entité est à l'intérieur de l'écran
    insideScreen :: a -> Bool

-- Loi vérifiée comme précondition avant chaque move.
law_positive_screenSpeed :: a -> ScreenScrollingSpeed -> Bool
law_positive_screenSpeed _ s = s > 0
```

La *typeclass* `Movable` sert surtout à pouvoir avoir une opération commune de mouvement, ainsi que la capacité à pouvoir filtrer selon l'emplacement dans l'écran ou non. Pour

insideScreen, les implémentations ne sont pas strictes : certaines instances peuvent être considérées dans l'écran même en étant au-dessus (c'est le cas par exemple des `Enemy`, apparaissant au-dessus de l'écran, et n'évoluant que dans la direction vers le bas. L'utilité est alors lors du filtrage de ne pas supprimer ce qui viennent d'apparaître).

La commutativité ne peut pas être conservée comme loi pour `move`, car le mouvement d'un `Object`, auquel chaque entité délègue la capacité à se déplacer, implique la possibilité de se déplacer indépendamment de la vitesse de défilement de l'écran donnée.

Typeclasses/Invariant.hs

```
class Invariant a where
    -- The invariant verified by 'a'
    prop_inv :: a -> Bool

-- Si l'invariant est vrai une fois, le vérifier à nouveau donne le même résultat.
law_invariant_stable :: Invariant a => a -> Property
law_invariant_stable x = prop_inv x ==> prop_inv x

-- Vérifier plusieurs fois l'invariant ne modifie pas sa valeur.
law_invariant_idempotent :: Invariant a => a -> Bool
law_invariant_idempotent x = prop_inv x == prop_inv x

-- Appliquer la fonction à une valeur valide préserve l'invariant.
-- Vérifié naturellement par mes tests QuickCheck sur les opérations,
-- qui contrôlent systématiquement l'invariant comme post-condition
law_invariant_preserved :: (Invariant a, Invariant b) => (a -> b) -> a -> Property
law_invariant_preserved f x = prop_inv x ==> prop_inv (f x)
```

La *typeclass* `Invariant` avait surtout été faite dans le but de pouvoir, à l'issue d'une opération sur un mur (`FiniteWall` / `InfiniteWall`), vérifier que les invariants des éléments contenus soient toujours vrais après application de l'opération.

Ses lois peuvent être considérées comme triviales, mais je ne savais pas quoi mettre d'autre. De même, peut être que cette *typeclass* n'est pas la plus pertinente.

Objects/Hitbox.hs

```
data Hitbox = Circle XCenter YCenter Radius
             | Rectangle XBottomLeft YBottomLeft Width Height
             | Hitboxes XCenter YCenter [Hitbox]
             deriving (Eq, Show)
```

> Permet de définir une `Hitbox` comme étant un cercle, un rectangle, ou une liste faite d'autres `Hitbox`. Utilisé par `Object` pour définir son emplacement spatial.
Instance de `Invariant`.

Objects/Objects.hs

```
class (Movable a) => Collidable a where
    -- Récupère une liste d'objets représentant l'entité
    -- (en général un seul objet)
    getObjects :: a -> [Object]
    -- Détecte s'il y a une collision, grâce aux positions actuelles des objets
    collision :: Collidable b => a -> b -> Bool
    -- Détecte s'il y a une collision après le déplacement de l'entité.
    -- Float correspond à la vitesse de défilement de l'écran
    willCollide :: Collidable b => a -> b -> ScreenScrollingSpeed -> Bool

-- Un objet doit toujours entrer en collision avec lui-même.
law_collidable_reflexive :: Collidable a => a -> Bool
law_collidable_reflexive x = collision x x

-- La détection de collision doit être indépendante de l'ordre des arguments.
law_collidable_symmetric :: (Collidable a, Collidable b) => a -> b -> Bool
law_collidable_symmetric x y = collision x y == collision y x

-- La prédiction de collision future doit correspondre
-- à la collision après déplacement.
law_collidable_will_collide :: Collidable a => Collidable b => a -> b ->
ScreenScrollingSpeed -> Bool
law_collidable_will_collide x y s =
    willCollide x y s == collision (move x s) y
```

La *typeclass* `Collidable`, nécessitant d'avoir instancié `Movable`, sert surtout à requérir aux instances un comportement de mouvement homogène, permettant alors de détecter d'éventuelles collisions. Cela repose notamment sur l'utilisation de `Object`, représentant spatialement les mouvements et positions des entités.

Les implémentations concrètes étant toutes identiques, le seule véritable pertinence de tests est alors sur les mouvements et vérification de collision directement chez l'instance `Object`.

```
data Direction = Direction Int Int deriving (Eq, Show)
```

> Définit un vecteur de direction.

Instance de Invariant.

```
newtype ObjectSpeed = ObjectSpeed Float deriving (Eq, Show)
```

> Définit la vitesse d'un **Object** pouvant bouger de manière indépendante.

Instance de Invariant.

```
data Object = Movable0 Hitbox Direction ObjectSpeed
    | Static0 Hitbox
    deriving (Eq, Show)
```

> Permet de définir un **Object** comme étant soit déplaçable par lui-même, indépendamment de la vitesse de l'écran et avec sa propre direction, soit statique, se déplaçant alors toujours vers le bas selon la vitesse de l'écran.

Combiné avec la **Hitbox** contenue, **Object** est la **brique de base des mouvements de mes entités** : tout ce qui est **Movable** contient nécessairement un **Object** auquel est **délégué les mouvements de l'entité**.

Instance de Invariant, Movable, Collidable.

Graphics/Assets.hs

```
class Renderable a where
    -- Récupère une liste d'assets traduits représentant l'entité
    getTranslatedAssets :: GameAssets -> a -> [Picture]

-- Garantit que le nombre d'assets affichés reste raisonnable.
law_renderable_finite :: Renderable a => GameAssets -> a -> Bool
law_renderable_finite ga x = Prelude.length (getTranslatedAssets ga x) < 100
```

La *typeclass* `Renderable` requiert une opération permettant de renvoyer une représentation graphique de l'entité (dont le nombre d'assets est raisonnable). Typiquement tout ce qui est affiché à l'écran instancie cette classe.

```
data GameAssets = GameAssets {
    -- players
    p1Pic :: Picture,
    p2Pic :: Picture,
    p1ExplosionPics :: Seq Picture,
    p2ExplosionPics :: Seq Picture,
    ...
} deriving Show
```

> Stocke tous les assets du jeu (sauf pour le `Background`), initialisé seulement au début du jeu.

```
data HealthBarAssets = HealthBarAssets {
    healthPics :: Seq Picture -- taille de 11
} deriving Show
```

> Stocke les assets des barres de vies des joueurs du jeu, contenu dans `GameAssets`.

```
data DigitAssets = DigitAssets {
    digits :: Seq Picture, -- taille of 10
    letterPPic :: Picture,
    digit1BlackPic :: Picture,
    digit2BlackPic :: Picture,
    letterPBlackPic :: Picture
} deriving Show
```

> Stocke les assets des chiffres du jeu, affichés en bas, contenu dans `GameAssets`.

Graphics/Background.hs

```
data Background = Background{
    backgroundPicture :: Picture, -- l'asset d'arrière plan
    backgroundScrollingSpeed :: ScreenScrollingSpeed,
    backgroundY :: YCoord -- le Y de l'asset central de l'arrière plan
```

```
} deriving (Eq, Show)
```

> Représente l'arrière-plan, ainsi que la position de la `backgroundPicture` centrale de l'arrière-plan (au rendu, un autre asset est mis au-dessus, et un autre en-dessous). Son défilement est indépendant du jeu.

Instance de `Invariant`, `Renderable`.

Graphics/Explosion.hs

```
data Explosion = Explosion {  
    explosionX :: XCoord, -- X centre de l'explosion  
    explosionY :: YCoord, -- Y centre de l'explosion  
    explosionPhaseCounter :: FrameCounter, -- compteur de frames de  
                                           l'animation d'explosion  
    explosionPhase :: AnimationPhase -- la phase d'explosion actuelle  
} deriving (Show, Eq)
```

> Représente l'animation d'une explosion après la collision d'un `Projectile` avec une autre entité.

Instance de `Invariant`, `Renderable`.

GameState/Rock.hs

```
data Rock =
  LeftRock {
    rockObject :: Object,
    rockAsset  :: RockAsset,
    rockForward :: Bool
  }
  | RightRock {
    rockObject :: Object,
    rockAsset  :: RockAsset,
    rockForward :: Bool
  } deriving (Show, Eq)
```

> Représente un rocher, utilisé comme faisant partie d'un mur du jeu. Il est déplacé selon le défilement de l'écran.

Instance de Invariant, Renderable, Movable, Collidable.

GameState/Wall.hs

```
newtype FiniteWall a = FiniteWall [a] deriving (Eq, Show)
```

> Représente un mur fini d'éléments **Collidable**, non vide, et chacun collisionnent avec son suivant. **FiniteWall Rock** est concrètement utilisé et se déplace alors selon le défilement de l'écran.

Instance de Functor, Foldable, Invariant, Renderable, Movable, Collidable.

```
newtype InfiniteWall a = InfiniteWall [a]
instance Eq a => Eq (InfiniteWall a) where
  (==) :: InfiniteWall a -> InfiniteWall a -> Bool
  (InfiniteWall xs) == (InfiniteWall ys)
    = (take nbTakeInfiniteWalls xs) == (take nbTakeInfiniteWalls ys)
instance Show a => Show (InfiniteWall a) where
  show :: InfiniteWall a -> String
  show (InfiniteWall xs) =
    "InfiniteWall " ++ show (take nbTakeInfiniteWalls xs) ++ "..."
```

> Représente un mur infini d'éléments **Collidable**. **InfiniteWall Rock** est concrètement utilisé et se déplace alors selon le défilement de l'écran.

Instance de Functor, Invariant, Renderable, Movable, Collidable.

```
data GameWalls = GameWalls {
  gameLeftWall :: InfiniteWall Rock,
  gameBackgroundLeftWall :: InfiniteWall Rock,
  gameRightWall :: InfiniteWall Rock,
  gameBackgroundRightWall :: InfiniteWall Rock,
  gameFiniteWalls :: [FiniteWall Rock] -- autre murs que les infinis par
défaut
} deriving (Eq, Show)
```

> Représente l'ensemble des murs du jeu. Les infinis sont initialisés au début de la partie, tandis que les finis sont générés et ajoutés durant la partie. Les murs contenus se déplacent tous selon le défilement de l'écran.

Instance de Invariant, Renderable, Movable, Collidable.

GameState/Projectile.hs

```
data Projectile =  
    PlayerShot Object ProjectileAsset Damage PlayerId  
    | EnemyShot Object ProjectileAsset Damage  
    deriving (Eq, Show)
```

> Représente un projectile, soit appartenant à un **Player**, soit à un **Enemy**. Dans le jeu, ces projectiles se déplacent avec leur propre vitesse et direction, constantes.

Instance de Invariant, Renderable, Movable, Collidable.

GameState/Bonus.hs

```
data Bonus = PlayerBonus {  
    bonusObject :: Object,  
    getBonus :: PlayerShootBonus -- le bonus contenu  
} deriving (Eq, Show)
```

> Représente un bonus lorsqu'il est sur l'écran du jeu, et se déplace avec le défilement de l'écran.

Instance de Invariant, Renderable, Movable, Collidable.

```
data PlayerShootBonus = NoBonus | ShootFaster | DelayDecreased | MoreDamages |  
    BiggerShots deriving (Eq, Show)
```

> Représente un bonus de tir pour le joueur.

Par défaut au début du jeu, le **Player** a un NoBonus.

GameState/Player.hs

```
data Player =  
    AliveP Object PlayerId Lifes Health Score PlayerShootBonus FrameCounter  
    FrameCounter  
    | InvincibleP Object PlayerId Lifes Health Score PlayerShootBonus  
    FrameCounter FrameCounter  
    | DeadP Object PlayerId Score FrameCounter AnimationPhase  
    deriving (Eq, Show)
```

> Représente un joueur. 3 différents cas :

- **Le joueur est vivant**, dans son état normal : celui-ci est alors affiché, en rouge si il a pris des dégâts récemment sans avoir perdu de vie, et peut tirer au bout d'un certain délai (qui peut être réduit par un des **PlayerShootBonus**).
- **Le joueur est vivant et temporairement invincible** (immédiatement après avoir perdu s'il lui en reste encore) : celui-ci est alors affiché en clignotant en blanc pendant 2 secondes, et peut tirer au bout d'un certain délai (qui peut être réduit par un des **PlayerShootBonus**).
- **Le joueur est mort** (n'a plus de vies) : celui-ci n'est plus affiché et ne peut pas tirer.

Instance de Invariant, Renderable, Movable, Collidable, Damageable.

GameState/Enemy.hs

```
data Enemy = Enemy {
    enemyObject :: Object,
    enemyHealth :: Health,
    enemyCollisionDamage :: Damage,
    enemyScoreGiven :: Score,
    enemyScript :: EnemyScript -- le script définissant le comportement
} deriving (Eq, Show)
```

> Représente un ennemi, dont le comportement est défini par son `EnemyScript`.

Instance de Invariant, Renderable, Movable, Collidable, Destroyable.

```
data EnemyScript =
    NoMoveButBoomEnemy
  | LeftRightShootEnemy{
        xTargetEnemyScript :: XCoord,
        yTargetEnemyScript :: YCoord,
        shootDelayEnemyScript :: Int
    }
  | LoopEnemy{
        yTargetEnemyScript :: Float,
        bottomLeftStepsEnemyScript :: NbStepsRemaining,
        leftStepsEnemyScript :: NbStepsRemaining,
        topLeftStepsEnemyScript :: NbStepsRemaining,
        topStepsEnemyScript :: NbStepsRemaining,
        topRightStepsEnemyScript :: NbStepsRemaining,
        rightStepsEnemyScript :: NbStepsRemaining,
        bottomRightStepsEnemyScript :: NbStepsRemaining,
        goBottomEnemyScript :: Bool
    }
    deriving (Eq, Show)
```

> Représente le comportement d'un ennemi, 3 cas :

- Pour **NoMoveButBoomEnemy**, pas de comportement spécifique.
- Pour **LeftRightShootEnemy**, celui-ci descend jusqu'à une coordonnée Y visée dans l'écran, et une fois cette coordonnée atteinte, si sa coordonnée X visée (aussi dans l'écran) est à sa gauche, il se déplace vers la gauche, et sinon il se déplace vers la droite. Quand il atteint le X visé, le signe du X change de façon à ce qu'il se dirige ensuite vers le côté opposé. De plus, il tire chaque seconde grâce au compteur de frames maintenu en interne.
- Pour **LoopEnemy**, celui-ci descend jusqu'à une coordonnée Y visée dans l'écran (avec une direction vers le bas), puis effectue un mouvement de boucle : sa direction est d'abord vers bas-gauche, puis gauche, puis haut-gauche, ... jusqu'à se rediriger vers le bas. Il continue alors jusqu'à sortir de l'écran et être supprimé par une des fonctions de la boucle principale du jeu.

Instance de Invariant.

GameState/Game.hs

```
data Game = Game {  
    keyboard :: Keyboard,  
    state :: GameState, -- StartMenuOption ou InGameInfos  
    assets :: GameAssets,  
    background :: Background,  
    frameCounter :: FrameCounter  
} deriving Show
```

> Représente le jeu, avec l'état du clavier, l'état du jeu **au sens de si l'on est sur le menu d'accueil ou en jeu**, les assets, l'arrière-plan et le compteur de frames (utilisé pour *timer* les générations périodiques de murs).

Instance de Invariant.

```
data GameState =  
    StartMenu StartMenuOption  
    | InGame InGameInfos  
    deriving (Eq, Show)
```

> Indique si l'on est sur le menu d'accueil, ou en pleine partie. **Pas une monad State.**

Instance de Invariant.

```
data StartMenuOption = OnePlayer | TwoPlayers deriving (Eq, Show)
```

> Indique l'option couramment considérée sur le menu d'accueil (celle entourée des chevrons).

Instance de Renderable.

```
data InGameInfos = InGameInfos {  
    gameScreenSpeed :: ScreenScrollingSpeed,  
    gamePlayer1 :: Player,  
    gamePlayer2 :: Player,  
    gameEnemies :: [Enemy],  
    gameWalls :: GameWalls,  
    gameProjectiles :: [Projectile],  
    gameHitExplosions :: [Explosion],  
    gameBonuses :: [Bonus]  
} deriving (Eq, Show)
```

> Représente **toutes les entités couramment dans la partie**, ainsi que la vitesse de déplacement de l'écran. Correspond donc à l'état global d'une partie, et est manipulé par la monade `InGameState`.

Instance de Invariant, Renderable.

```
type InGameState a = St.State InGameInfos a
```

> Représente **la monade State du jeu**, instance de State (à la fin du fichier).

Boucle principale du jeu

Voici ma boucle principale de jeu, appelée à chaque frame depuis le *app/main.hs*, et prenant notamment en **paramètres** le **compteur de frames**, un **générateur construit spécifiquement à cette frame** (utilisé et mis à jours lors de mes générations aléatoires), **l'état actuel du clavier**, ainsi qu'un flottant correspondant au **temps écoulé entre deux frames** :

```
updateInGame :: FrameCounter -> StdGen -> Keyboard -> Float -> InGameState ()
updateInGame nbFrames gen kbd deltaTime = do
    igi <- St.get

    -- mise à jour des animations des joueurs (vaisseau rouge en cas de
    -- dégâts, ou explosion de mort)
    runPlayersAnimationSt

    -- génération potentielle d'un mur fini aléatoire
    gen2 <- generateWallSt nbFrames gen

    -- génération potentielle d'ennemis
    gen3 <- generateEnemiesSt nbFrames gen2

    -- les ennemis tirent et se déplacent
    runEnemiesSt

    -- déplacement des murs
    moveWallsSt

    -- déplacement des bonus
    moveBonusesSt

    let p1Dead = isPlayerDead (gamePlayer1 igi)
        p2Dead = isPlayerDead (gamePlayer2 igi)

    if not p1Dead
    then do
        -- éjecte le joueur 1 des murs si nécessaire
        bumpPlayerFromWallsSt True

        -- mise à jour de la direction et de la vitesse du joueur 1
        updatePlayerDirectionSpeedSt True (player1NewDirectionSpeed kbd
            deltaTime)
```

```

-- déplacement du joueur 1 dans les limites de l'écran
movePlayerInsideScreenSt True

-- gestion des collisions entre le joueur 1 et les ennemis
handleCollisionPlayerWithEnemiesSt True

-- gestion des collisions entre le joueur 1 et les bonus
handleCollisionPlayerWithBonusesSt True
else return ()

if not p2Dead
then do
    -- éjecte le joueur 2 des murs si nécessaire
    bumpPlayerFromWallsSt False

    -- mise à jour de la direction et de la vitesse du joueur 2
    updatePlayerDirectionSpeedSt False (player2NewDirectionSpeed kbd
        deltaTime)

    -- déplacement du joueur 2 dans les limites de l'écran
    movePlayerInsideScreenSt False

    -- gestion des collisions entre le joueur 2 et les ennemis
    handleCollisionPlayerWithEnemiesSt False

    -- gestion des collisions entre le joueur 2 et les bonus
    handleCollisionPlayerWithBonusesSt False
else return ()

-- mise à jour des animations d'explosion des projectiles touchés
runExplosionsSt

-- déplacement des projectiles
moveProjectilesSt

-- gestion des collisions des projectiles, avec éventuellement génération
  de bonus lorsque des ennemis meurent
_ <- handleCollisionProjectilesWithPlayersEnemiesSt gen3

-- incrémentation des compteurs de délai de tir des joueurs encore en vie
incrementPlayersShootFrameCountersSt

```


Extensions implémentées

Extensions basiques

En terme d'extensions, j'ai tout d'abord implémenté toutes les fonctionnalités basiques du jeu :

- Déplacements des entités, selon le défilement de l'écran ou indépendamment.
- Gestion des collisions entre les entités.
- Gestion du cycle de vie du joueur.
- Murs finis et infinis, avec notamment le fait d'être poussé et éventuellement déplacé si nécessaire lors d'une collision.
- Ennemis avec différents comportements.
- Prise en compte des projectiles, que ce soit des joueurs et des ennemis.

Point pertinent : gestion du repoussement par un mur

Concernant l'implémentation complète, je vous invite à vous reporter aux fonctions `bumpPlayerFromWalls` et `keepBumpin` dans **src/GameState/Game.hs**.

Toute l'utilité de ma typeclass `Collidable`, ainsi que de mon implémentation avec `Object` statique ou déplaçable, y sont démontrées. En commençant avec `bumpPlayerFromWall`, une première étape consiste à vérifier si une collision a lieu actuellement entre le joueur concerné par l'appel à la fonction, et un des murs du jeu. Mes données `Player` et `GameWalls` étant des instances de `Collidable`, il me suffit alors de faire appel à la fonction de collision, avec ces deux éléments en paramètres :

```
if not (collision p gw) then ((), igi)
```

Si la conditions s'avère être fausse, alors il nous faut pousser le joueur vers le bas, et surtout s'assurer que celui-ci ne soit pas une nouvelle fois en collision après déplacement : le déplacement d'un joueur n'étant pas lié à la vitesse de défilement de l'écran, on va alors artificiellement créer un `Object` statique avec la hitbox du joueur, et on va tester avec cet `Object`, instance également de `Collidable`, si celui-ci, obéissant au défilement vers le bas de l'écran avec son implémentation de mouvement d'objet statique, va collisionner ou non après mouvement vertical (et en s'assurant être encore dans les limites de l'écran) :

```
po = playerObject p
h = objectHitbox po
staticPo = initStaticObject h
...
if (bottomYPlayer-ss) >= bottomYScreenWithBarBound && not (willCollide
    staticPo anotherP ss) then
...
```

Si après cette étape le repoussement vertical mène encore à une collision, alors on passe au repoussement horizontal avec un appel à la fonction `keepBumpin`. L'idée est dans un premier temps de se générer des coordonnées X candidates, alternant à la gauche et à la droite du joueur, et étant de plus en plus éloignées de celui-ci :

```

candidates = concatMap (\distance ->
  let offset = (fromIntegral distance) * ss
      xLeft = xStart - offset
      xRight = xStart + offset
  in [xLeft, xRight]
) [1..1000::Int]

```

Le but est alors d'itérer sur le tableau des X candidats jusqu'à trouver la bonne position, sans collision, en passant par la fonction interne :

```

findValidPos :: [Float] -> Object
findValidPos (x:xs) =
  let objAtX = initPlayerObject x y (objectDirection obj)
      (objectSpeed obj)
  -- Plus de collision, on renvoie l'Object du joueur à ce X
  in if x >= leftBound && x <= rightBound && not (collision objAtX
      gw) && not (collision objAtX anotherP)
  then objAtX
  else findValidPos xs

```

Là encore, notre fonction de collision étant appelée entre un [Object](#), un [GameWalls](#) et un [Player](#).

Cet exemple sert surtout à montrer la flexibilité de ma *typeclass* [Collidable](#) : je peux l'utiliser avant une éventuelle collision, avec `willCollide`, ou directement avec `collide`, entre différents types de données ([Player](#), [GameWalls](#), [Object](#)) de manière uniforme, sans duplication de code.

Point pertinent : suivi du comportement d'un ennemi

Une chose particulièrement importante dans ma réalisation des comportements des ennemis a été de suivre le bon déroulement de leur position durant leur script. Cela s'effectue via la `prop_post_moveEnemy` présente dans **src/GameState/Enemy**, et est particulièrement importante car elle vérifie que l'ensemble des champs des scripts restent cohérents après chaque mouvement.

La post-condition étant vraiment très longue, il est difficile pour moi de la présenter ici, mais je vous invite à aller la regarder car celle-ci permet vraiment de garantir le bon déroulement des patterns suivis par mes ennemis.

Extensions avancées

En termes d'extensions avancées, voici mes principaux choix et grandes lignes d'implémentation :

Génération procédurale du jeu

Le déroulement de ma partie ne suit pas de script global, à la place je génère de manière périodique et/ou aléatoire de nouvelles entités. Comme montré précédemment, j'ai une fonction d'initialisation pour mes murs infinis en début de partie, mais j'ai également une fonction de génération de mur fini, de manière aléatoire :

```
-- Crée un mur fini de Rocks de manière aléatoire
startFiniteWall :: StdGen -> (FiniteWall Rock, StdGen)
startFiniteWall gen =
    let
        -- Maximum de 30 rochers dans le mur généré
        (nbRocks, gen1) = randomR (1::Int, 30) gen

        -- Centre X de ce mur
        (xCenter, gen2) = randomR ((leftXScreenBound + 300)::Float,
                                   (rightXScreenBound - 300)) gen1
        leftSide = xCenter <= 0

        -- Largeur maximale parmi tous les assets de Rocks
        maxW = maximum (toList widthRocks)

        -- Dépassement horizontal maximal autorisé autour du centre X
        overflow = (maxW/2) * 0.6
        (lowerX, upperX) = (xCenter - overflow, xCenter + overflow)

        -- Coordonnée Y de départ
        baseY = topYScreenBound + 100

        -- Suite finie de coordonnées Y pour les segments du mur (Rocks)
        ys = map (\i -> baseY + fromIntegral i * 15) ([0 .. nbRocks - 1] ::
                                                       [Int])

        -- Index aléatoires des assets de Rocks
        randomWalls = randomRs (0, nbRockAssets - 1) gen2

        -- Positions X aléatoires des rochers, pouvant être hors écran
        randomX = randomRs (lowerX, upperX) gen2
```

```

-- Associe chaque index d'asset de rocher à une position X aléatoire
randomValues = zip randomWalls randomX

in
  (initFiniteWall (zipWith (makeWallObject leftSide) ys randomValues),
   gen2)

where
  -- Crée un Rock individuel faisant partie du mur fini
  -- Premier argument : côté gauche ou droit du mur
  -- Deuxième argument : position Y
  -- Troisième argument : paire (index d'asset Rock, position X)
  makeWallObject :: Bool -> YCoord -> (Int, XCoord) -> Rock
  makeWallObject leftSide y (numRock, x) =
    startInitRock x y numRock leftSide True

```

L'idée étant de générer un mur fini n'ayant que des éléments se chevauchant, et surtout renvoyant un nouveau générateur. Ce point est particulièrement important pour maintenir une génération aléatoire non biaisée, car au contraire si mes fonctions de générations aléatoires, utilisées durant le jeu, utilisaient à chaque fois le même générateur, cela conduirait à des séquences répétitives ou corrélées, ce qui casserait l'indépendance statistique attendue entre les événements générés.

Ainsi, la génération reste *pure*, tout en simulant un comportement aléatoire fiable et non biaisé au cours d'un tour de la partie, ce qui est très important dans le cadre de la génération procédurale.

Un autre point qui peut être pertinent est l'instanciation de générateurs dans le cadre de tests unitaires. Personnellement, ce que j'ai fait est simple : **pour tous les tests unitaires d'une même fichier, lorsqu'un générateur est nécessaire, celui-ci est toujours instancié avec une seed fixée** : cela permet donc entre les lancement de faire en sorte que les résultats des tests ne puissent pas être influencé par le générateur.

Jeu à 2 joueurs coopératif

Une autre extension importante que j'ai effectué est la gestion coopérative du jeu à 2 joueurs. Pour m'adapter à cela, j'ai décidé de garder mes types de données de joueurs tels qu'actuellement, et de plutôt ajouter un paramètre booléen à mes fonctions permettant alors d'identifier, dans le cadre de la mise à jour de l'état global du jeu dans **src/GameState/Game**, si je dois traiter le joueur 1 ou le joueur 2.

Un point particulièrement intéressant a été néanmoins la gestion des collisions entre un joueur et son environnement, incluant donc l'autre joueur. Dans la fonction `movePlayerInsideScreen`, je dois donc effectuer 3 étapes :

- Gérer une éventuelle collision avec un mur
- Gérer une éventuelle collision avec un autre joueur

- Gérer une éventuelle collision avec une bordure de l'écran (par délégation à `movePlayer`)

Le tout n'ayant aucune incidence sur les autres éléments de mon jeu, ni par exemple sur la vie de mon joueur, j'ai alors ces pré-conditions et post-conditions :

```
prop_pre_movePlayerInsideScreen :: Bool -> InGameInfos -> Bool
prop_pre_movePlayerInsideScreen isP1 (InGameInfos _ p1 p2 _ _ _ _) =
    let p = if isP1 then p1 else p2
    in not (isPlayerDead p)

prop_post_movePlayerInsideScreen :: Bool -> InGameInfos -> Bool
prop_post_movePlayerInsideScreen isP1 igi@(InGameInfos _ _ _ _ _ _) =
    let (_, igi'@(InGameInfos _ p1' p2' _ _ _ _)) = movePlayerInsideScreen
    isP1 igi
    in (sameInGameInfosExceptPlayer isP1 igi igi') &&
        if isP1
        then not (isPlayerDead p1')
        else not (isPlayerDead p2')
```

Ici, l'appel à `sameInGameInfosExceptPlayer` permet de s'assurer que, mis à part mon joueur, toutes les autres infos de l'état de jeu n'ont pas changé. Ce qui est donc particulièrement important, c'est que ma gestion des collisions n'impacte pas le reste.

Invincibilité temporaire sur perte de vie

Une extension importante a aussi été l'introduction d'une période d'invincibilité pour un joueur venant de perdre une vie (si il lui en reste encore après).

Pour cela, je me suis doté d'un constructeur supplémentaire pour mon type de joueur, traduisant l'état remarquable qu'est la période d'invincibilité :

```
data Player = ...
    | InvincibleP Object PlayerId Lifes Health Score PlayerShootBonus
    ...
```

Une autre chose pertinente a été de se procurer des fonctions de transitions d'état :

Dans le sens vivant > invincible :

```
aliveToInvinciblePlayer :: Player -> Player
aliveToInvinciblePlayer (AliveP po pId lifes _ score psb frameShootCpt _) =
    (initInvinciblePlayer po pId (lifes-1) 100 score psb frameShootCpt 1)
aliveToInvinciblePlayer _ = error "only an alive player can become invincible"
```

et au contraire invincible > vivant :

```
invincibleToAlivePlayer :: Player -> Player
invincibleToAlivePlayer (InvincibleP po pId lifes health score psb
frameShootCpt _) =
    (initAlivePlayer po pId lifes health score psb frameShootCpt 0)
invincibleToAlivePlayer _ = error "only an invincible player can become alive"
```

Cette dernière fonction étant appelée à l'issue de la période d'invincibilité (comptée à l'aide d'un compteur de frames maintenu en attribut interne).

Des chose qui me paraissent être intéressante à montrer, sont par exemple les pré et post-conditions du premier sens de changement :

```
prop_pre_aliveToInvinciblePlayer :: Player -> Bool
prop_pre_aliveToInvinciblePlayer (AliveP _ _ lifes health _ _ _) = lifes > 1
&& health == 0
prop_pre_aliveToInvinciblePlayer _ = False

prop_post_aliveToInvinciblePlayer :: Player -> Bool
prop_post_aliveToInvinciblePlayer p =
    let p' = aliveToInvinciblePlayer p
    in case (p, p') of
        ((AliveP po pId lifes _ score psb frameShootCpt _),
         (InvincibleP po' pId' lifes' health' score' psb' frameShootCpt'
          frameCpt'))
            -> lifes' == (lifes-1) && health' == 100 && po == po' && pId ==
                pId' && score == score' && psb == psb'
                && frameShootCpt == frameShootCpt' && frameCpt' == 1
        ((AliveP _ _ _ _ _ _ _), _) -> False
        (_, _) -> True
```

Pour devenir invincible : un joueur doit avoir au préalable plus qu'une vie (pour décrémenter et être ensuite forcément à 1 ou plus), et doit avoir exactement plus aucun point de vie. Naturellement, la précondition vérifie aussi qu'un joueur invincible ou mort ne puisse pas devenir invincible.

Concernant la post-condition, il nous faut vérifier alors que le nouvellement invincible joueur ait un nombre de vies décrémenté, des points de vie réinitialisés à 100, et que les autres champs restent identiques par rapport à avant l'invincibilité, sauf pour l'initialisation du compteur de frames de la période d'invincibilité à 1.

Si par contre j'ai en entrée un joueur vivant (vérifiant donc la pré-condition), et en sortie un autre joueur qu'un invincible, alors la post-condition est nécessairement violée.

Bonus de tir pour le joueur

Enfin, une dernière extension qui me paraissait être pertinente est l'introduction de bonus : ceux-ci actuellement ne concernent que les tirs des joueurs.

Un bonus est représenté sur l'écran par le type `Bonus` défini dans `src/GameState/Bonus.hs`. Celui-ci doit naturellement suivre le défilement de l'écran, et implique donc dans son invariant le fait d'être caractérisé par un `Object` statique.

Le bonus en lui-même est contenu dans un autre attribut, accessible par `getBonus`.

Un point intéressant peut être la génération de ces bonus sur l'ancien emplacement d'ennemis :

```
generateNewBonuses :: StdGen -> [Enemy] -> [Enemy] -> (StdGen, [Bonus])
generateNewBonuses gen enemiesBefore enemiesAfter =
```

```
  let
    -- Recherche des ennemis morts
    -- (présents dans enemiesBefore mais absents de enemiesAfter)
    deadEnemies = filter (\e1 ->
      let h1 = objectHitbox (enemyObject e1)
          (ex1, ey1) = centerHitbox h1
      in not (any (\e2 ->
        let h2 = objectHitbox (enemyObject e2)
            (ex2, ey2) = centerHitbox h2
        in ex1 == ex2 && ey1 == ey2
        ) enemiesAfter
      )
    ) enemiesBefore

    -- Pour chaque ennemi mort, génère éventuellement un bonus
    (finalGen, bonuses) =
      foldr generateBonusForEnemy (gen, []) deadEnemies

  in (finalGen, bonuses)
```

```
prop_pre_generateNewBonuses :: StdGen -> [Enemy] -> [Enemy] -> Bool
prop_pre_generateNewBonuses _ enemiesBefore enemiesAfter =
  length enemiesAfter <= length enemiesBefore
```

```
prop_post_generateNewBonuses :: StdGen -> [Enemy] -> [Enemy] -> Bool
prop_post_generateNewBonuses gen enemiesBefore enemiesAfter =
```

```

let (_, bonuses) = generateNewBonuses gen enemiesBefore enemiesAfter
    deadEnemies = filter (\e -> e `notElem` enemiesAfter) enemiesBefore

in
    -- Le nombre de bonus est au plus égal au nombre d'ennemis morts
    length bonuses <= length deadEnemies

    -- Chaque position de bonus correspond à la position
    -- d'un ennemi mort
    && all (\bonus ->
        let (bx, by) =
            centerHitbox (objectHitbox (bonusObject bonus))
        in any (\enemy ->
            let (ex, ey) =
                centerHitbox (objectHitbox (enemyObject enemy))
            in bx == ex && by == ey
        ) deadEnemies
    ) bonuses

    -- Chaque bonus possède un type valide
    -- (différent de NoBonus)
    && all (\bonus ->
        case getBonus bonus of
            NoBonus -> False
            _ -> True
    ) bonuses

```

Le plus intéressant concerne la post-condition :

- Il faut nous assurer que le nombre de bonus générés est au plus le nombre d'ennemis morts (la différence de tailles entre les 2 listes prises en paramètres).
- Il faut vérifier que chaque bonus est à l'emplacement d'un ennemi mort.
- Il faut vérifier que le bonus contenu généré est bien valide.